



AWS Academy Cloud Developing
Module 06 Student Guide
Version 2.0.6

200-ACCDEV-20-EN-SG

© 2024, Amazon Web Services, Inc. or its affiliates. All rights reserved.

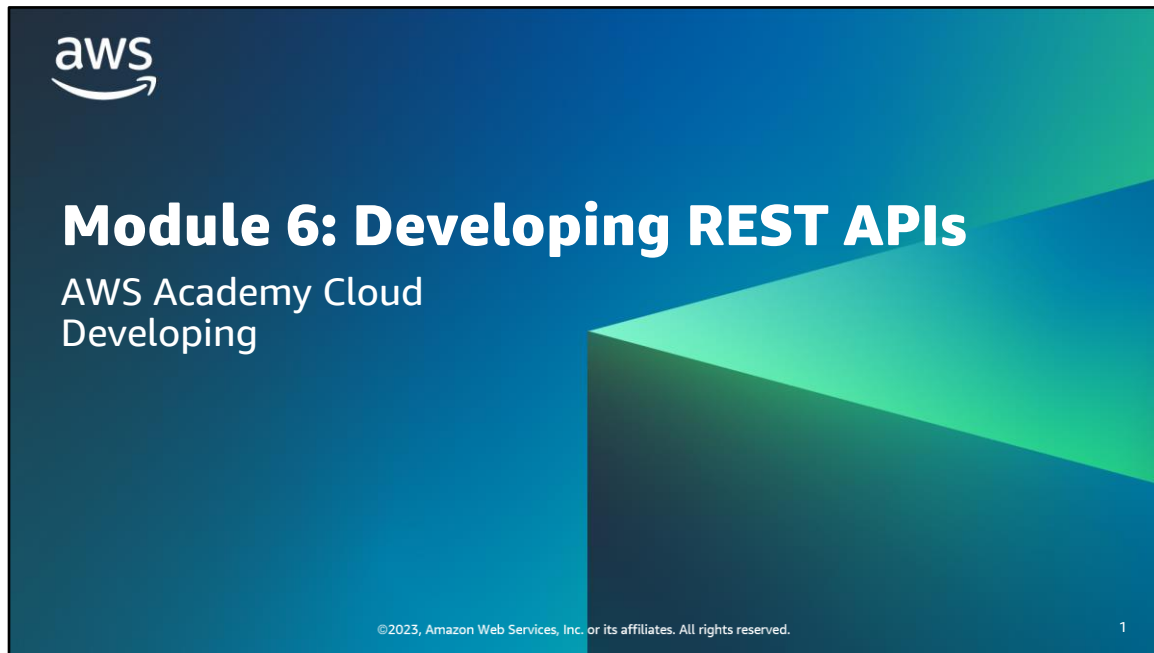
This work may not be reproduced or redistributed, in whole or in part,
without prior written permission from Amazon Web Services, Inc.
Commercial copying, lending, or selling is prohibited.

All trademarks are the property of their owners.

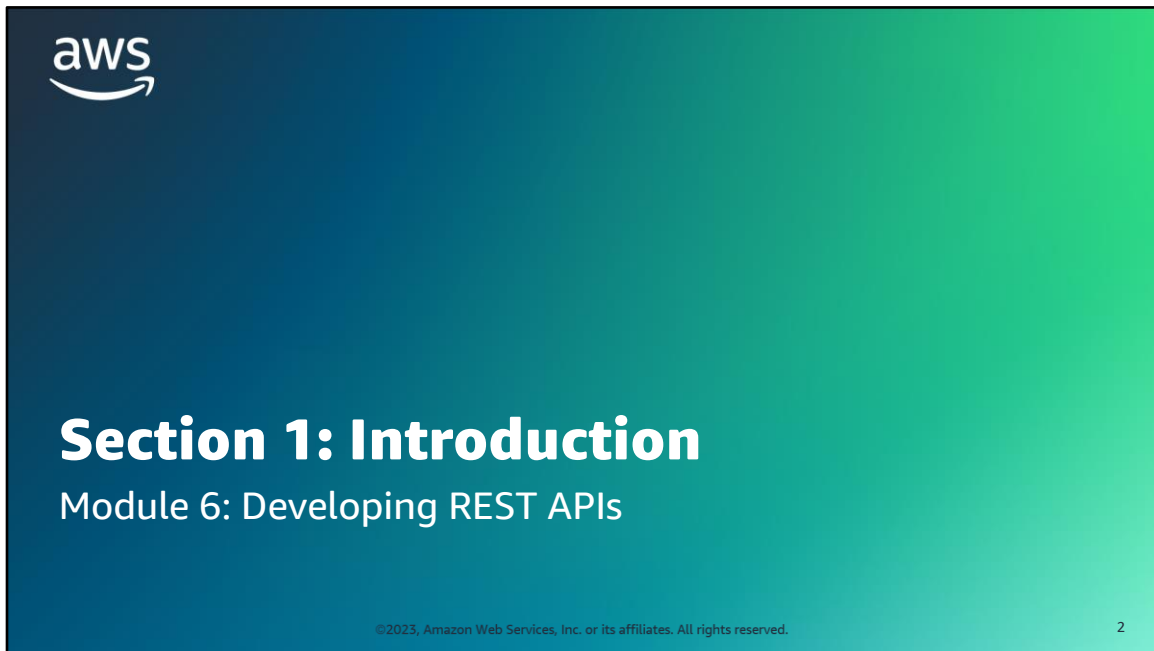
Contents

[Module 6: Developing REST APIs](#)

4



Welcome to Module 6: Developing REST APIs.



Section 1: Introduction.

Module objectives

At the end of this module, you should be able to do the following:

- Recognize APIs
- Describe Amazon API Gateway
- Indicate the steps for developing REST APIs with API Gateway
- Use API Gateway to create, publish, maintain, monitor, and secure APIs



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

3

At the end of this module, you should be able to do the following:

- Recognize APIs
- Describe Amazon API Gateway
- Indicate the steps for developing REST APIs with API Gateway
- Use API Gateway to create, publish, maintain, monitor, and secure APIs

Module overview

Sections

1. Introduction
2. Introducing APIs
3. Introducing API Gateway
4. Creating a REST API
5. Integrating with API Gateway
6. Deploying an API
7. Controlling access to a REST API
8. Monitoring a REST API
9. Optimizing API Gateway

Lab

Developing REST APIs with Amazon API Gateway



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

4

This module includes the following sections:

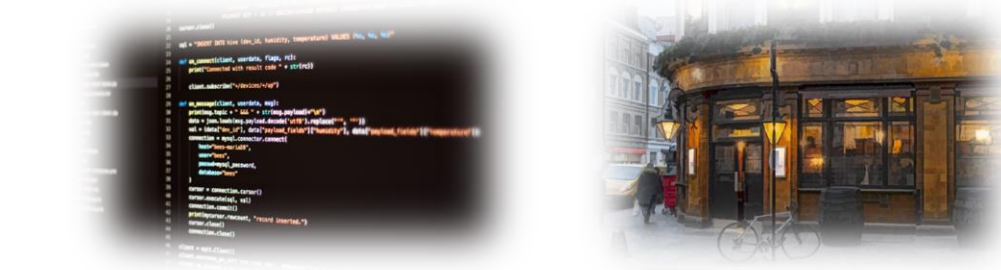
1. Introduction
2. Introducing APIs
3. Introducing API Gateway
4. Creating a REST API
5. Integrating with API Gateway
6. Deploying an API
7. Controlling access to a REST API
8. Monitoring a REST API
9. Optimizing API Gateway

This module also includes a lab about developing REST APIs with API Gateway.

Finally, you are asked to complete a knowledge check that will test your understanding of key concepts that this module covers.

Café business requirement

Product information for the café is now being stored in a database, and Sofía wants to use an API to integrate the website with the database. She wants to test the front end and get approval from Frank and Martha before completing the full integration.



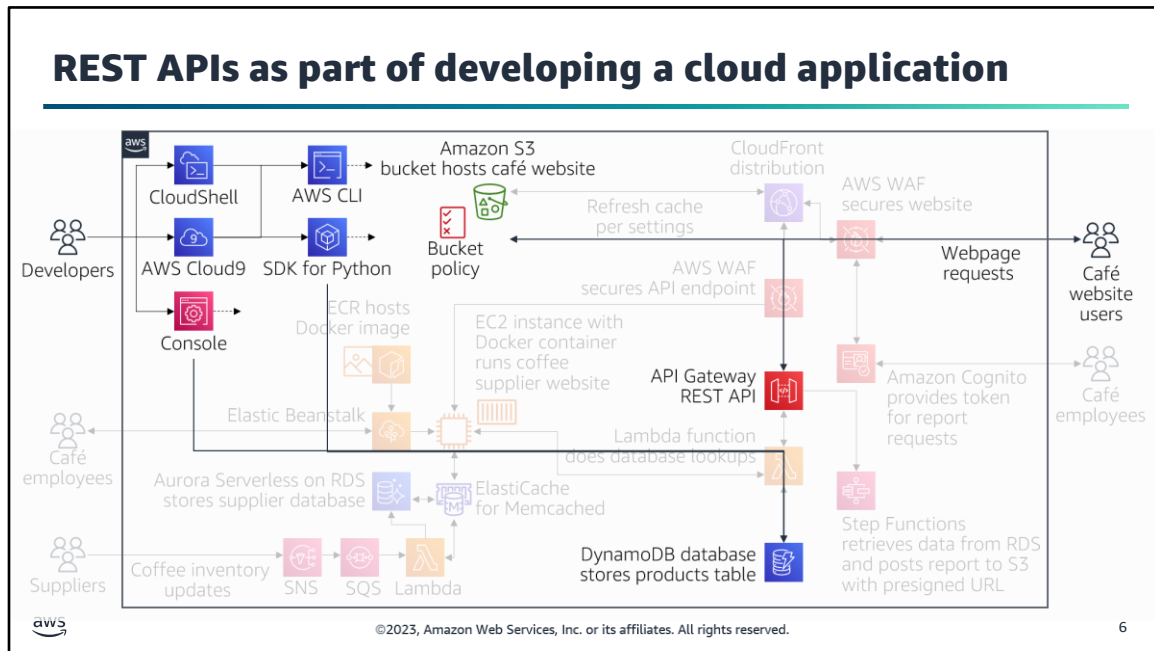
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

5

Product information for the café is now being stored in a database, and Sofía wants to use an API to integrate the website with the database. She wants to test the front end, and get approval from Frank and Martha before completing the full integration.

To use an API to access the data in the Amazon DynamoDB table, Sofía needs to modify the website to use an API Gateway endpoint.

Before she implements the production integration to DynamoDB, Sofía plans to show Nikhil how to test updates, and she wants to get approval from the owners. She can use a mock endpoint in API Gateway to quickly create and test the API, and demonstrate it for the owners. After she tests and gets approval, she can update the backend integration to connect to the DynamoDB table.



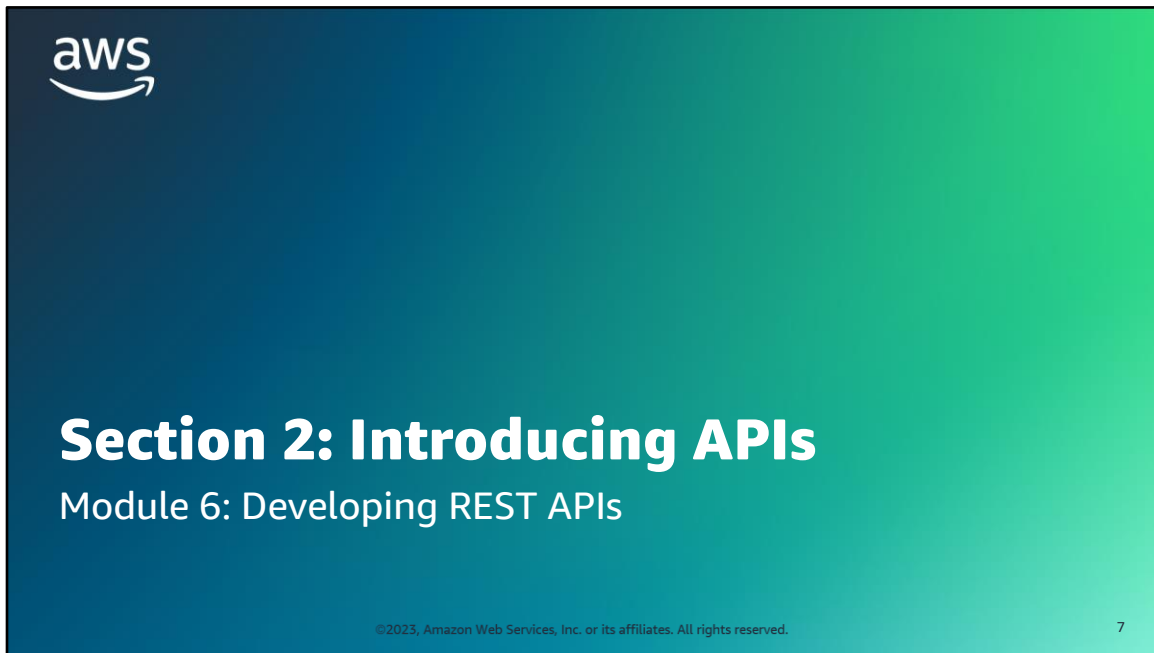
The diagram on this slide gives an overview of the application that you will build through the labs in this course. The highlighted portions are relevant to this module.

As highlighted in the diagram, you will use API Gateway endpoints to enable the website to request and retrieve data through an API. You will use mock endpoints to test the front end integration rather than integrating with the database.

You should be familiar with these terms used throughout the module:

REST = Representational State Transfer

API = application programming interface.



Section 2: Introducing APIs.

What is an API?

An API is a software mechanism that simplifies development by doing the following:



Abstracting
implementation
details



Exposing only
objects or actions
that a developer
needs



Establishing how an
information
provider and an
information user
communicate

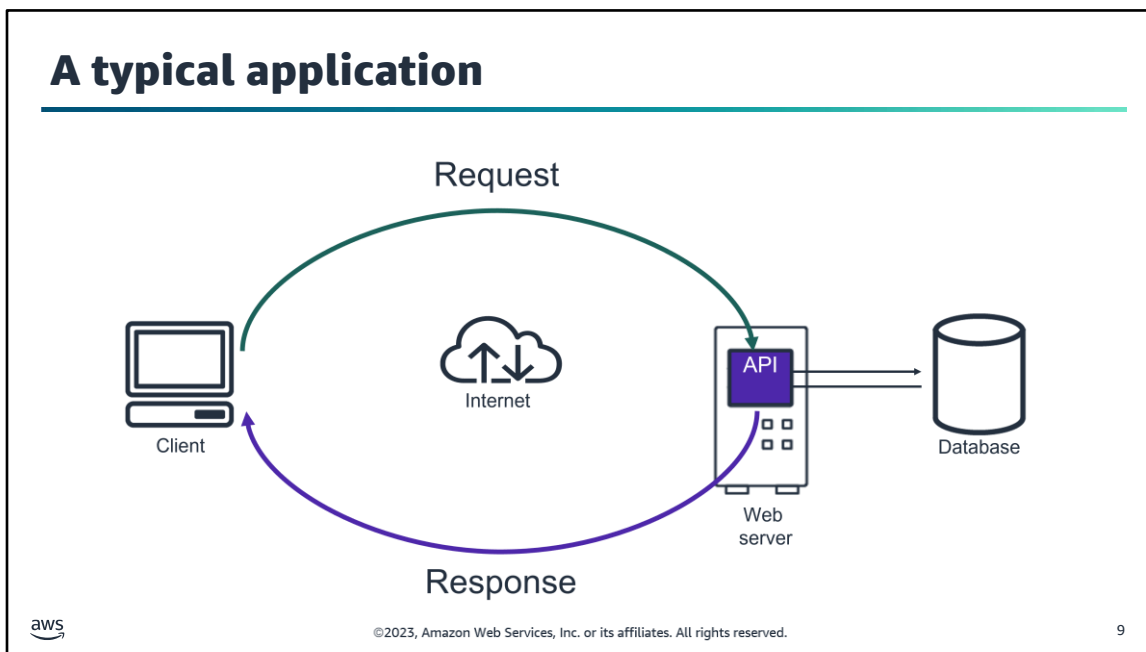


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

8

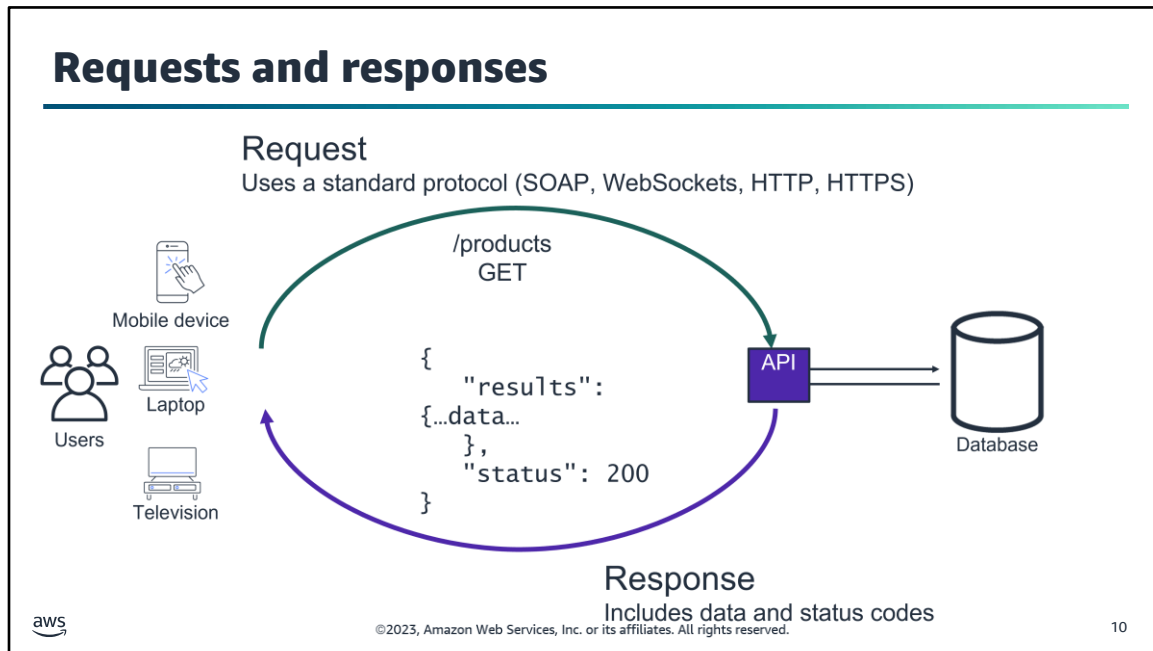
An API is a software mechanism that simplifies development by doing the following:

- Abstracting implementation details
- Exposing only objects or actions that a developer needs
- Establishing how an information provider and an information user communicate



In a typical web application, clients send requests and get responses from an API that resides on a web server. The API provides the interface to application resources such as databases.

Web services offer APIs for developers to use. For example, social networks like Facebook or Twitter and payment processing services like Amazon Pay and PayPal provide helpful APIs for developers. By using the APIs, developers can more easily write code that works with those applications.



API requests use a standard protocol like SOAP (an XML-based protocol), WebSockets (a bidirectional protocol), or HTTP. A request from a device reaches your server, and your server responds with a status code and potentially some data.

Whether a client application runs as a website or a native application (iOS, Android), it usually gives some dynamic data back to the client application.

This process is usually carried out through an HTTP application protocol that humans can read, and is commonplace across the internet. Thus, developing for heterogeneous systems is relatively easy. Even in cases where a client application is using SOAP, it would still usually transmit requests by using HTTP.

WebSockets and server-side, event-driven applications are becoming more popular with the advent of *real-time* applications, and WebSockets use a different (bidirectional) protocol. WebSockets use URLs that start with *ws://* instead of *http://*.

With any protocol, it is a best practice to use the secure variants, for example *HTTPS* (or *SHTTP*) and *WSS*.

Response status codes fall into a few main categories:

- Response status codes with *20x*, such as 200 and 201, are successful responses.
- Status codes with *40x*, such as 403 or 412, are telling the requesting client that they must do something differently. For example, they passed an incorrect password or did not pass the information that the server needed to fulfill the request.
- Status codes in the *50x* range, such as 500 or 501, are responses to the client that something went wrong that was not their fault. An example might be that the server encountered a programming error or resource failure during handling of the request.

What is a RESTful API?

Representational State Transfer (REST)

- Architectural style
- Standard way of structuring requests from a client to a server

RESTful API

- An API that adheres to REST principles



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

11

You often see an API that is categorized as either a RESTful API or a WebSocket API, but this wording is an oversimplification.

Representational State Transfer (REST) is purely an architectural style, and APIs that you build that adhere to REST principles are called RESTful APIs.

REST is the standard way of structuring requests from a client to a server.

Non-RESTful APIs that use HTTP are valid and quite common.

Section 2 key takeaways



- APIs simplify development by abstracting implementation details and providing a shared definition of how to communicate with an application.
- Clients send requests to an API and get a response back.
- REST is a standard way to structure API requests.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

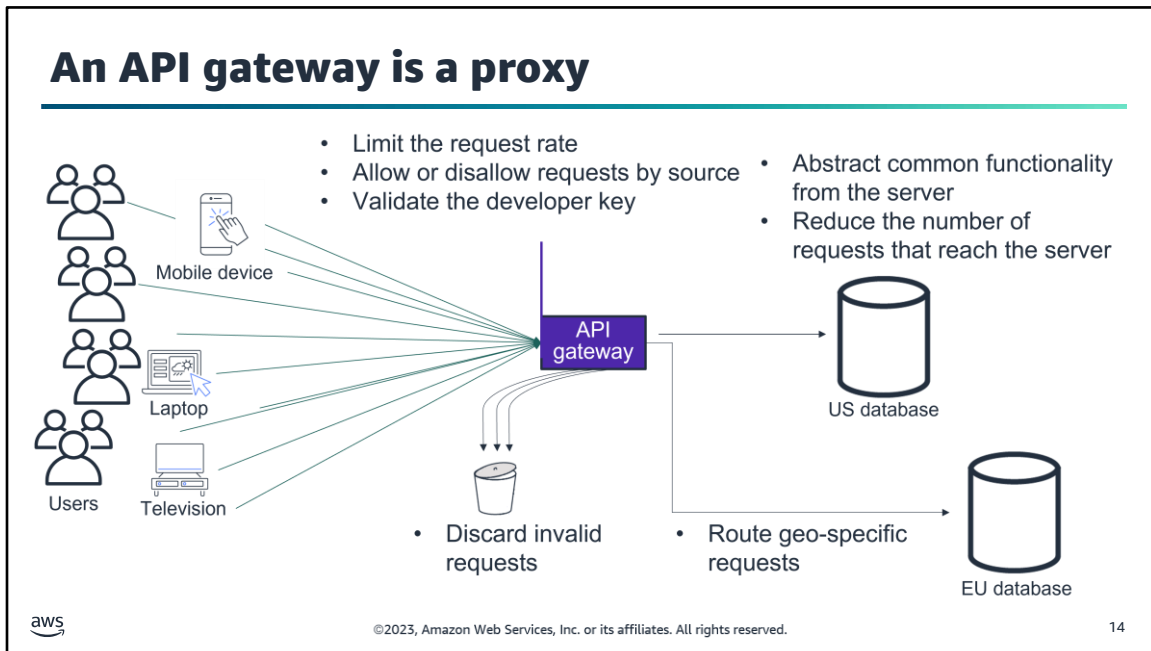
12

Some key takeaways from this section of the module include:

- APIs simplify development by abstracting implementation details and providing a shared definition of how to communicate with an application.
- Clients send requests to an API and get a response back.
- REST is a standard way to structure API requests.



Section 3: Introducing API Gateway.

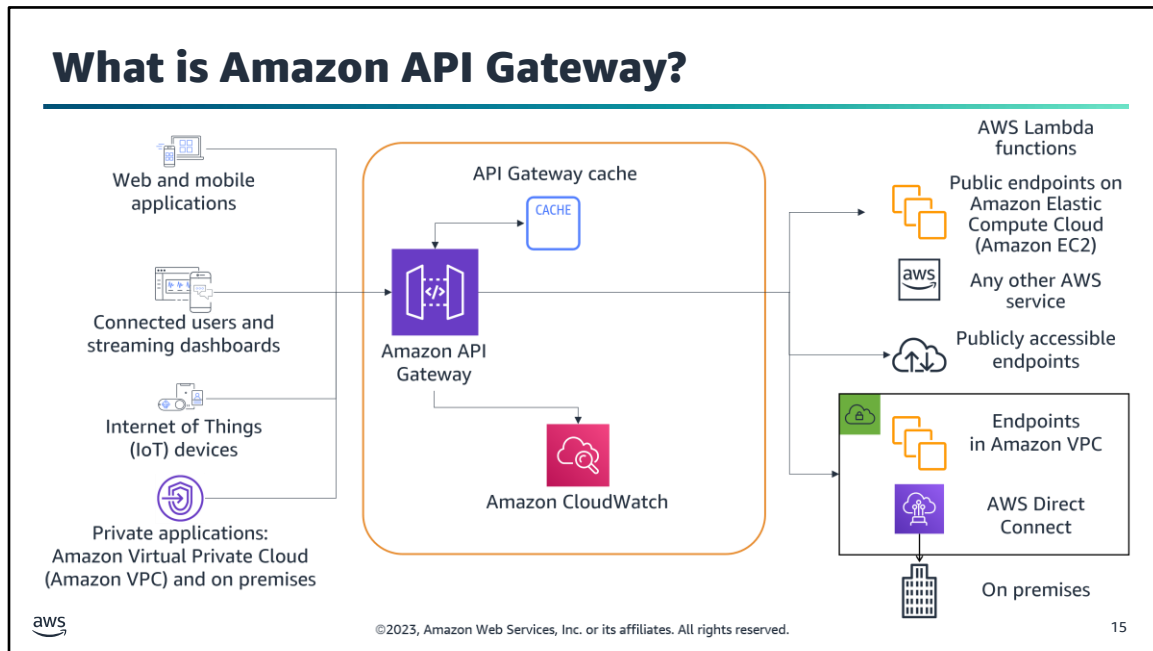


An API gateway is a proxy that is between your client and server (or servers behind a load balancer). Its task is to handle most of the common problems that developers encounter when managing dynamic server-driven applications.

Without an API gateway (proxy) between the client and the server, you would need to programmatically handle everything at the server. This arrangement would not be a good use of the server's processing capabilities. It is better to abstract some core common functionality out of the server and position it as a standalone proxy.

For example, you can have the proxy:

- Rate limit the application, so that the server is not overwhelmed. It is similar to a doorman in a night club.
- Ensure that the request contains the required developer keys. Perhaps you have an API that is designed for various developers or sites to consume. Examples are Twitter or Google Maps, where you get an API key. The proxy can validate requests and drop invalid requests before they even get to your server.
- Limit access based on language headers or geographical regions. In some cases, it might reroute to the correct server for that locale.



Amazon API Gateway is a fully managed, serverless service that developers can use to easily create, publish, maintain, monitor, and secure APIs at any scale. API Gateway acts as the *front door* for applications to access data, business logic, or functionality from your backend services.

API Gateway can easily be linked to other AWS services. For example, you might not even want to maintain a server. Instead, you might choose to *go serverless*. API Gateway can intercept requests from clients and massage and adjust the data if needed before forwarding information to AWS Lambda, for example. Then, while returning data, it can potentially massage and adjust the data from Lambda before sending it back to the client.

You might want to use Amazon API Gateway instead of having client applications that connect directly to DynamoDB, which gives you more control and flexibility.

You can create both RESTful APIs and WebSocket APIs with API Gateway. You pay only for the API calls that you receive and the amount of data that is transferred out.

For more information about how API Gateway works, see “Amazon API Gateway” at <https://aws.amazon.com/api-gateway/>.

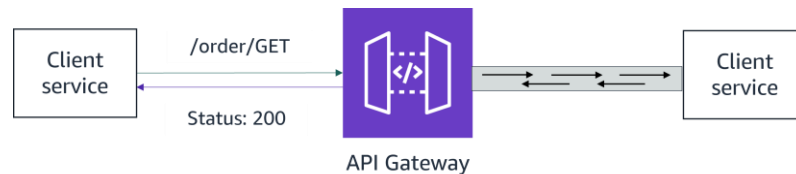
API Gateway supported protocols

RESTful

- Request/response
- HTTP methods (GET, POST)
- Short-lived communication
- Stateless

WebSockets

- Two-way communication channel
- Long-lived communication
- Stateful

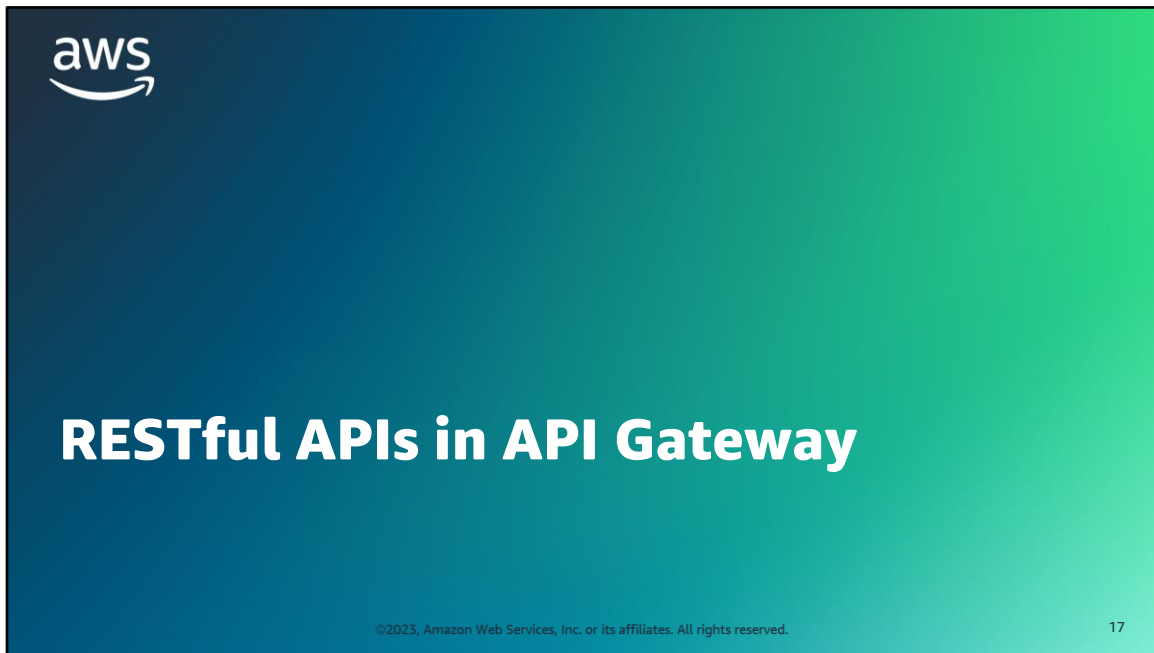


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

16

Not all HTTP requests that a client application generates are classified as RESTful. However, for simplicity, the term *RESTful* is used in this module to distinguish between WebSockets and HTTP.

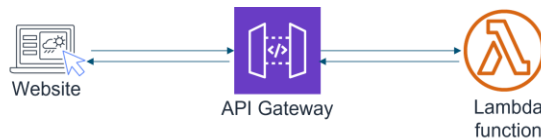
With REST APIs, a client services sends a request to an API endpoint , for example /order/GET. API Gateway returns a response, for example Status: 200 . With WebSockets APIs, a two-way communication channel opens between the client service and the API and information flows in both directions between them while the connection is active.



RESTful APIs in API Gateway

Two types of RESTful APIs in API Gateway

- REST API
- Gives the developer full control over API requests and responses
- Supports features that are not yet available in HTTP APIs
- HTTP API
- Simplifies the development of APIs that require only API proxy functionality
- Designed for the lowest cost and lowest latency



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

18

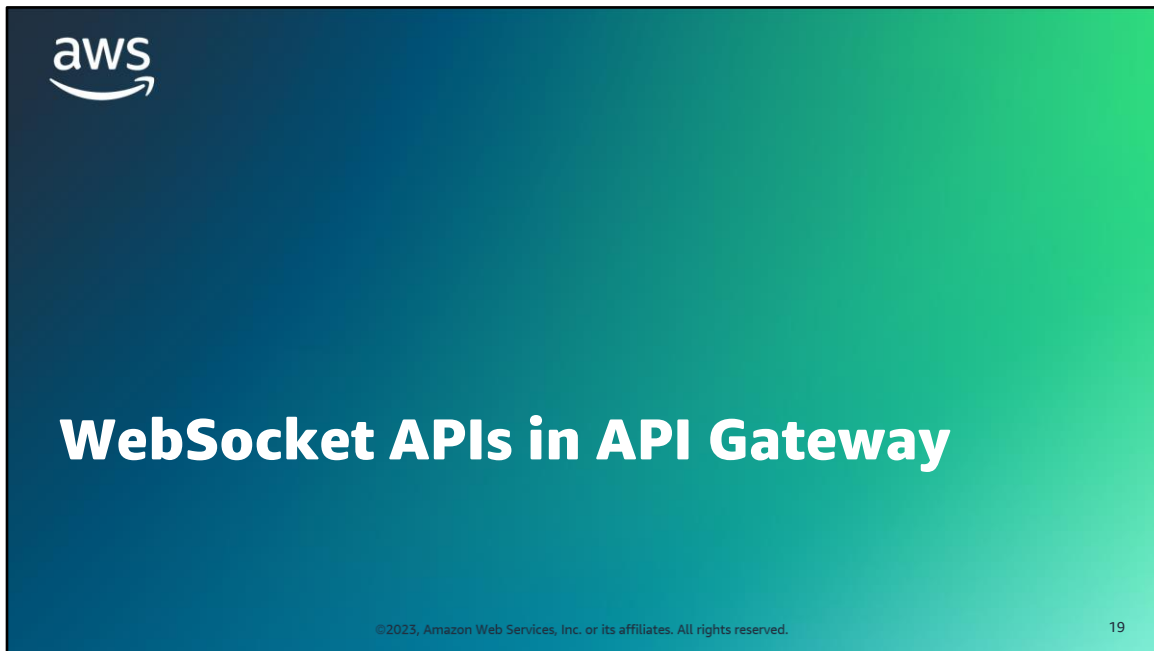
In some situations, API Gateway REST API features are more than you need. When you create a REST API in API Gateway, you are entering advanced mode. You can manipulate the requests that come in and go out, have fine-grained control over highly customized authentication options, and can set up mock endpoints. Other features will be discussed later in the course.

You might find that you don't need all of those features. For example, you might only need a proxy in front of a Lambda function. You can't directly access Lambda from client applications by using a standard HTTP request because Lambda endpoints are not publicly accessible URIs. You must either use an AWS software development kit (SDK) for that purpose, or put something in front of the function to take the request and transform it into a Lambda payload. You also need something to transform the response from Lambda on its way back to the client.

If you need API Gateway only for this purpose, use the API Gateway HTTP API type. HTTP APIs are lightweight, easy to use and set up, have lower latency, and cost less than the standard REST API. Savings can be up to 70% compared to the standard REST API.

As HTTP APIs mature, features currently available only with REST APIs will be added. Unless you need advanced features or need to monetize your API, choose the HTTP type for your APIs in API Gateway.

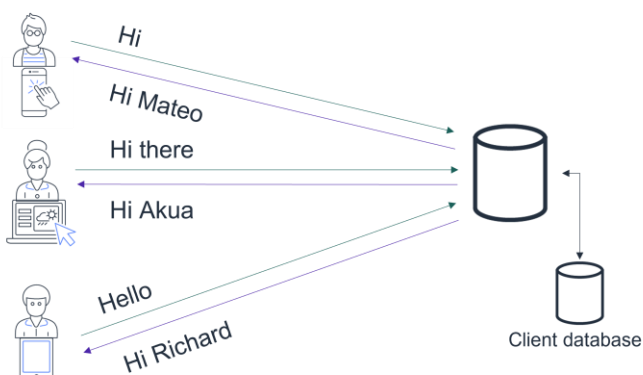
For the latest comparison of REST and HTTP API features, see “Choosing between HTTP APIs and REST APIs” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html>.



WebSocket APIs in API Gateway

WebSockets are bidirectional

Real-time, two-way communication

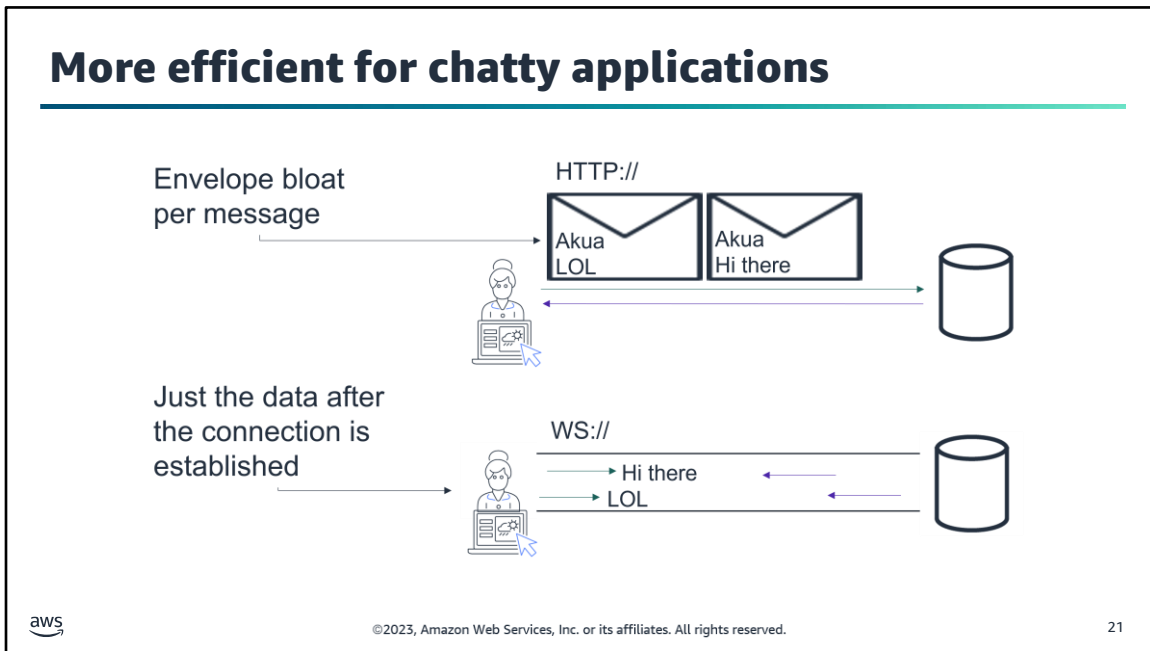


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

20

If you want to build real-time, two-way communication applications, such as chat apps and streaming dashboards, use WebSockets or server-sent events.

Without more detail, this image looks like standard HTTP requests that are going back and forth between clients. But with a closer look at the requests, you find the benefit of WebSockets for this type of communication.

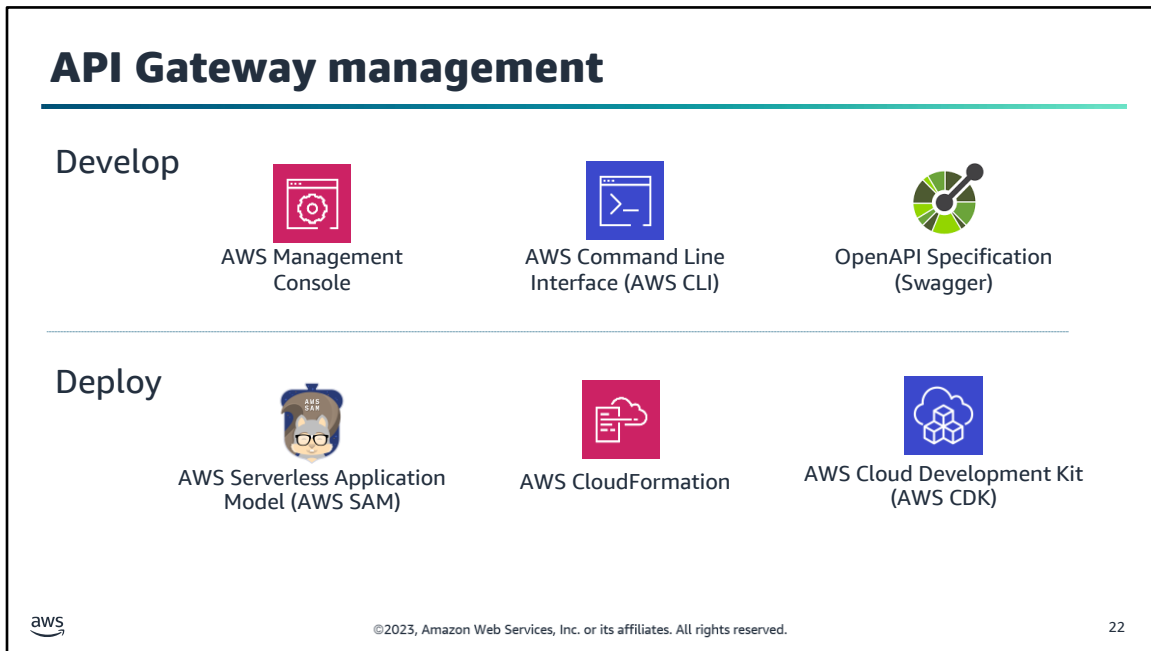


A typical HTTP request requires much ancillary information to function. It contains headers and other bits of information, such as connection bearer tokens for authentication. This is summarized in the first part of the illustration labeled as envelope bloat per message.

In the HTTP example, Akua first sends a simple message ("Hi there"). The actual data is small compared to the extraneous data that goes back and forth for every HTTP interaction. When she sends the second message ("LOL"), the HTTP overhead is repeated. It is unnecessary and costly to send header information for every interaction.

If an application is sending and receiving a large volume of messages (it is chatty), it is better to set up a two-way pipe between client and server. After the initial setup, both sides can send small efficient messages back and forth freely. WebSockets is a standard for setting up this two-way pipe, and it handles issues such as retries and broken connections. The second part of the illustration shows how both of Akua's messages ("Hi there" and "LOL") are sent within the two-way pipe without requiring additional information per message once the connection is established. You can also set your WebSockets routing to indicate which backend services should be invoked based on characteristics such as message content. Examples of such services include Lambda, Amazon Kinesis, or an HTTP endpoint.

You can use Secure WebSockets (WSS) for a secure, encrypted version of WebSockets (WS).



You can create and work with API Gateway APIs by using the AWS Management Console, AWS Command Line Interface (AWS CLI), Swagger, and OpenAPI to document and share your API definitions.

AWS provides several deployment frameworks that can be used to write infrastructure as code and deploy your APIs in the cloud.

The AWS Serverless Application Model (AWS SAM) and the AWS SAM CLI are particularly useful for locally testing and deploying APIs as part of a serverless application. AWS SAM is a superset of AWS CloudFormation but provides a simplified syntax for common serverless components.

You can use CloudFormation to write infrastructure as code that uses a common language. It supports YAML Ain't Markup Language (YAML) and JavaScript Object Notation (JSON) templates. CloudFormation automates the provisioning and ongoing updates of resources.

The AWS Cloud Development Kit (AWS CDK) uses the familiarity and expressive power of programming languages for modeling your applications. It provides a library of constructs that covers many AWS services and features. AWS CDK provisions resources with CloudFormation.

Section 3 key takeaways



- Amazon API Gateway is a fully managed service that provides API proxy and simplifies API development.
- API Gateway supports RESTful and WebSocket APIs.
- API Gateway integration with AWS services frees developers from writing integration code.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

23

Some key takeaways from this section of the module include:

- Amazon API Gateway is a fully managed service that provides API proxy and simplifies API development.
- API Gateway supports RESTful and WebSocket APIs.
- API Gateway integration with AWS services frees developers from writing integration code.



Section 4: Creating a REST API.

Creating a REST or HTTP API

Create an API in API Gateway:

- Use the AWS CLI
- Use the API Gateway console

Base structure for REST and HTTP APIs

`https://{restapi_id}.execute-api.{region}.amazonaws.com/{stage_name}/`



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

25

When you create and deploy a REST API (or the simpler and lower-cost HTTP API), API Gateway generates a URI. This URI has the components that are illustrated in this slide. You will learn more about stages later in this module.

You can create and invoke your APIs with the API Gateway console or the AWS CLI.

Example: Creating an HTTP API that is connected to a Lambda function

With the AWS CLI

```
aws apigatewayv2 create-api \  
    --name my-api \  
    --protocol-type HTTP \  
    --target  
arn:aws:lambda:us-east-  
2:123456789012:function:function-name
```

With the console

1. Open the API Gateway console.
2. Choose Build on the HTTP panel.
 - If any APIs already exist, the console displays a list of existing APIs and an option to create an API. Choose Create API. Then choose Build on the HTTP API panel.
3. Choose Add integration, and choose a Lambda function in the same Region as the API that you are creating.
4. Enter a name for your API.
5. Choose Review and Create.
6. Choose Create.

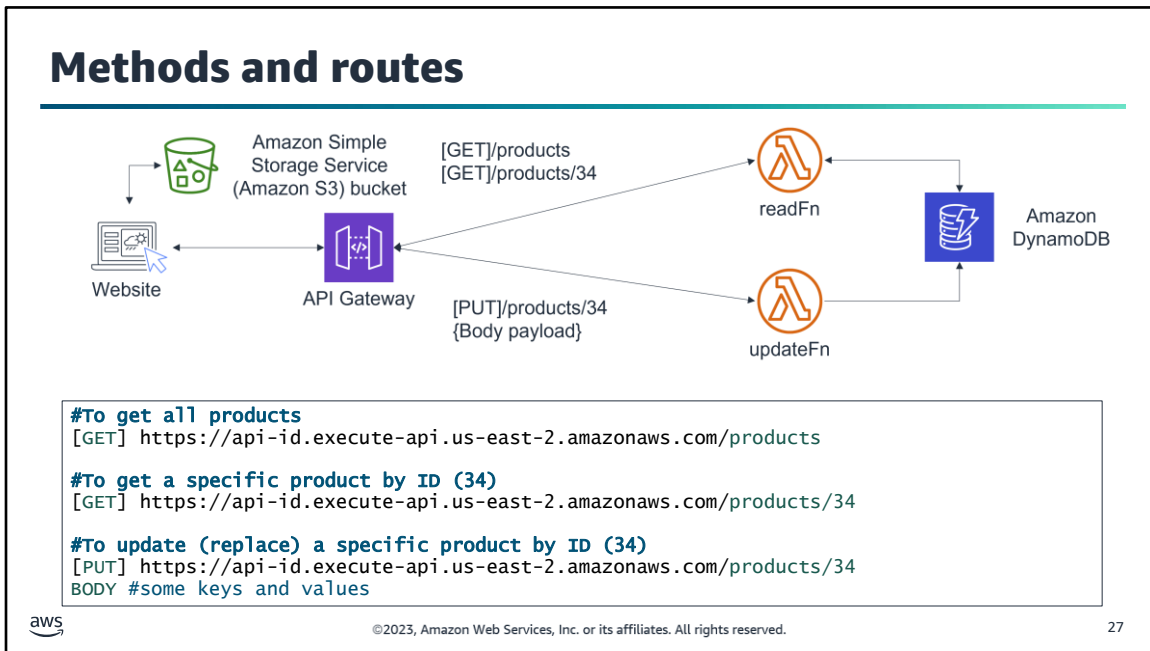


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

26

With either of these examples, the result is an HTTP API that is integrated with a Lambda function by using a default catchall route. It also results in a default stage that is configured to automatically deploy changes.

The steps for creating a basic REST API are similar, but some more work is necessary in configuring the API. For example, you must create at least one stage and deploy your API to it.



It is likely that you will have an API that does more than one task. Therefore, a default endpoint won't give you everything that you need.

For example, suppose that you are using API Gateway for a dynamic website. Static content (such as index.html, CSS, or JavaScript) might be hosted on an Amazon Simple Storage Service (Amazon S3) bucket and use specific paths to reach specific API methods to do specific things.

In this example, the default endpoint (`/products`) that gets all products, and the GET method for a specific product (34) both route to `readFn`. The PUT method routes to the `updateFn` function.

For the GET example, `readFn` is written so that it can handle receiving all of the products. For instance, if a variable of `product_id` is passed to the Lambda function, then it responds with a query instead of a scan.

Creating GET routes

```
aws apigatewayv2 create-route \  
    --api-id 03z501670a \  
    --route-key 'GET /products/{product_id}' \  
    --target arn:aws:lambda:us-east-2:123456789012:function:readFn  
  
#and for the "all" products  
aws apigatewayv2 create-route \  
    --api-id 03z501670a \  
    --route-key 'GET /products' \  
    --target arn:aws:lambda:us-east-2:123456789012:function:updateFn
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

28

This slide illustrates the code you that might use to create the GET routes for getting all products or getting a specific product.

Creating a PUT route

```
aws apigatewayv2 create-route \  
    --api-id 03z501670a \  
    --route-key 'PUT /product/{product_id}' \  
    --target arn:aws:lambda:us-east-2:123456789012:function:updateFn
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

29

This slide illustrates the code that you might use to create the PUT route to update a specific product. The product is identified by the `product_id` with the `updateFn` function.

Using the greedy path variable {proxy+}

```
aws apigatewayv2 create-route \  
  --api-id 03z501670a \  
  --route-key 'GET /products/{proxy+}' \  
  --target arn:aws:lambda:us-east-2:123456789012:function:readFn
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

30

Another alternative is to use the greedy path variable {proxy+}. With proxy+, you can set up a single Lambda integration that absorbs any nested paths that are included on the API call. With this approach, you let the Lambda function (or other server-based code) react to changes in any of the API resources under the default endpoint. (In this example, it is /products.) For example, a request like [GET] `https://api-id.execute-api.us-east-2.amazonaws.com/products/34/stuff/more/evenmore/stuff` would be passed to the readFn to be parsed and responded to with that function's logic.

This catchall approach is a simpler API to build, but in general, it is a better practice to use more specific paths.

Creating a WebSocket API

```
aws apigatewayv2 create-api \  
  --name "myWebSocketApi" \  
  --protocol-type WEBSOCKET \  
  --route-selection-expression '$request.body.action'
```

JSON
message

```
{  
  "service" : "chat",  
  "action" : "join",  
  "data" : {  
    "room" : "room1234"  
  }  
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

31

The command to create a WebSocket API is similar to the commands for REST and HTTP APIs.

The route select expression is required for WebSockets. It provides information that your server application will need.

For example, suppose that you want your API's behavior to be based on the action property of the JSON message that you are sending. You include the `.action` JSONPath expression in the route selection expression.

The action value in this example would evaluate to the word *join*, and your server code knows which method to use.

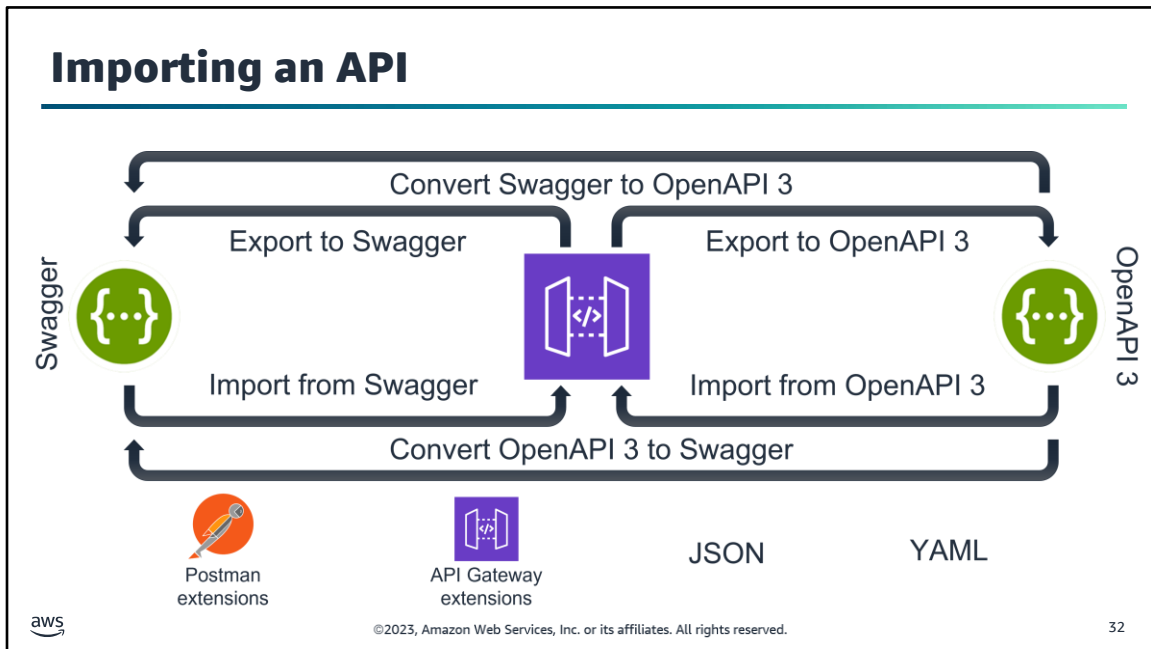
In addition to custom expressions, WebSockets needs three other route keys :

- `$connect`
- `$disconnect`
- `$default`

API Gateway calls the `$connect` route when a persistent connection between the client and a WebSocket API is being initiated.

API Gateway calls the `$disconnect` route when the client or the server disconnects from the API.

If the route selection expression cannot be evaluated against the message or if no matching route is found, API Gateway calls the `$default` route.



Many developers might be aware of Swagger, which has been rebadged as an OpenAPI specification. OpenAPI is a declarative definition file that defines how your API works. When you create APIs, you might want to provide documentation about how to use your API, especially if you are working with third-party developers.

You can also use an OpenAPI or Swagger file to build an API from an imported definition.

To import an OpenAPI definition for creating your API, use the following code:

```
aws apigatewayv2 import-api
  --body file://api-definition.json
```

To export your API definition, use:

```
aws apigateway get-export
  --parameters extensions='apigateway'
  --rest-api-id abcdefg123
  --stage-name dev
  --export-type swagger latestswagger2.json
```

The course does not go deeper into defining API definition files. However, for more information, see the “Amazon API Gateway Developer Guide section Working with OpenAPI Definitions for HTTP APIs” at

<https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-open-api.html>.

Section 4 key takeaways



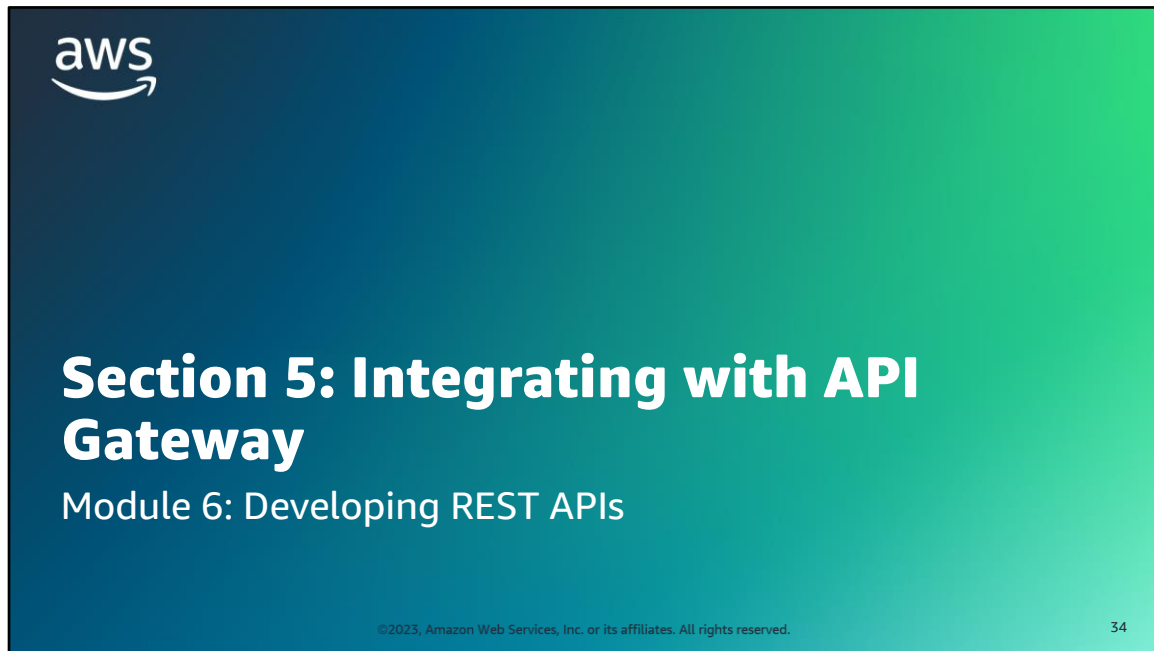
- You can create APIs by:
 - Using the AWS CLI
 - Using the API Gateway console
 - Importing an OpenAPI definition
- As a best practice, use different paths to perform different actions and connect to different backend integrations.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

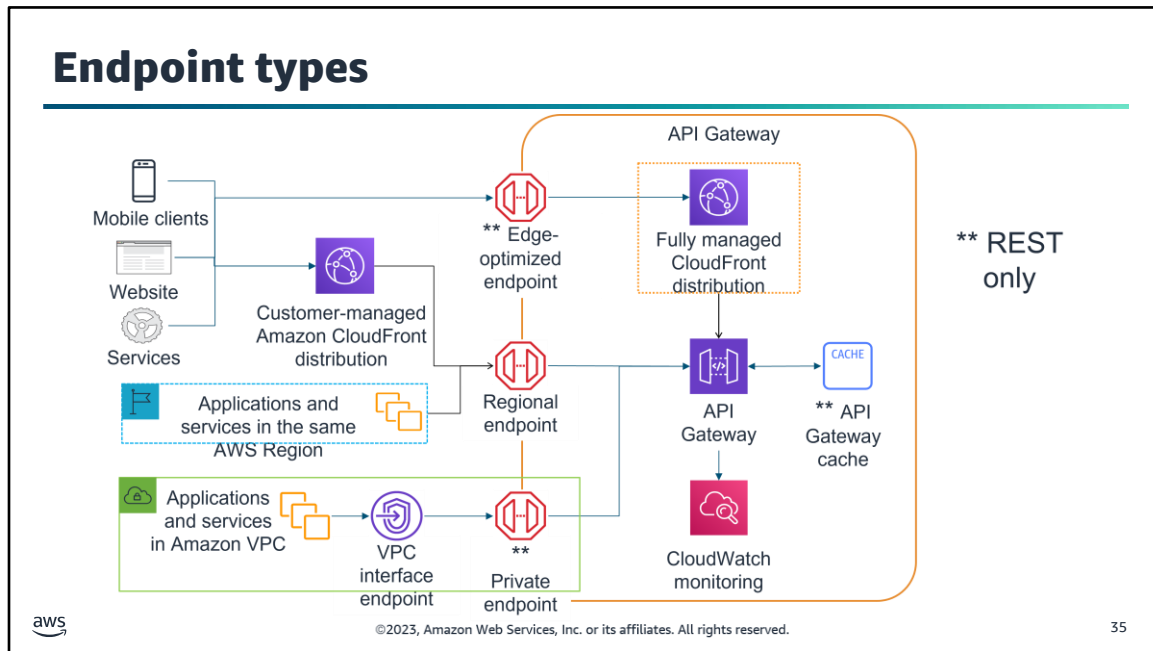
33

Some key takeaways from this section of the module include:

- You can create APIs by:
 - Using the AWS CLI
 - Using the API Gateway console
 - Importing an OpenAPI definition
- As a best practice, use different paths to perform different actions and connect to different backend integrations.



Section 5: Integrating with API Gateway.



This slide looks more closely at the Amazon API Gateway architecture that was described earlier in this module.

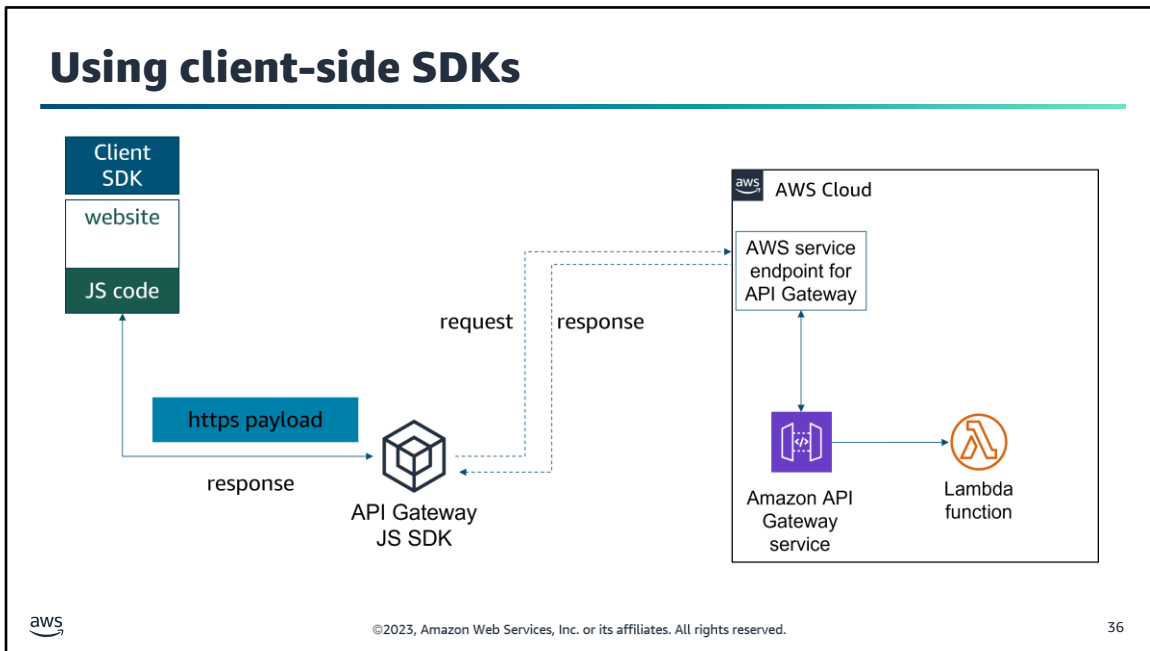
API Gateway handles all the tasks that are involved in accepting and processing API calls from three types of API endpoints: edge-optimized, Regional, and private.

With REST APIs, you have the option of an edge-optimized endpoint. With this endpoint type, API Gateway uses its own CloudFront distribution to reduce roundtrip time for your requests and responses. This endpoint type is designed for globally distributed clients. It gives you built-in distributed denial of service (DDoS) protection through its Amazon CloudFront distribution without your needing to set up a separate CloudFront distribution. This option is not available for HTTP or WebSocket APIs.

All of the API types (REST, HTTP, and WebSocket) support Regional endpoints. Regional endpoints are recommended for general use cases and are designed for building APIs for clients that are in the same AWS Region.

As the diagram illustrates, it is a best practice to set up a CloudFront distribution in front of Regional endpoints. These endpoints will be accessed by websites and other externally focused services. CloudFront helps to protect against DDoS attacks and to reduce the roundtrip time for requests and responses.

REST APIs also provide the option of private endpoints, which are accessible from only within your Amazon Virtual Private Cloud (Amazon VPC). This endpoint type is designed for building APIs that are used internally or by private microservices.



In general, you can write your API calls by using HTTPS requests to an API Gateway endpoint. Alternatively, you can use the API Gateway option to export an SDK for your defined API. Then, you can use that SDK with your client-side code to make calls to the API.

API Gateway generates custom SDKs for mobile app development with Android and iOS (Swift and Objective-C), and for web app development with JavaScript. API Gateway also supports generating SDKs for Ruby and Java. After an API and its models are defined in API Gateway, you can use the AWS Management Console to generate and download a client SDK. Alternatively, use the API Gateway APIs to do so. Client SDKs are generated only for REST APIs in API Gateway.

For example, suppose that you are using AJAX calls on your dynamic website. You could generate the JavaScript (JS) SDK for your API, and use the generated SDK to write calls to the API.

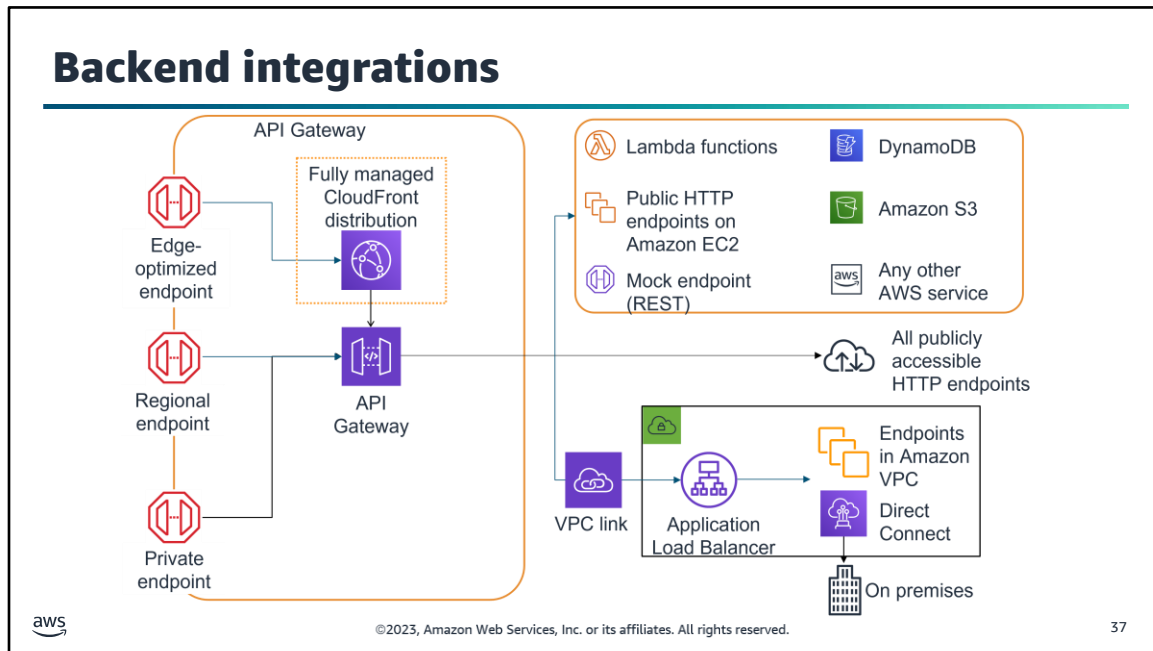
When you write your own integration this way, you must have the client send a securely signed payload to your AWS account. In this example, the client is a website that uses JavaScript (JS). Using the generated SDK, you send the securely signed payload to the AWS service endpoint for API Gateway. AWS can unpack that payload and deliver to the API gateway service what the service needs to respond to the call.

AWS awaits a response and then packages the response and sends a secure payload back to the SDK. The SDK converts that message into a response that the JS can work with.

Although the generated SDK for the API simplifies coding this integration, the code to build the request payload from the client-side SDK is non-trivial. Additionally, it forces you to work with the API credentials on the client side, which is a bad practice from a security standpoint.

For these reasons, the recommended approach is to use HTTPS requests to your API Gateway endpoint. This approach is simpler to code and more secure.

As a developer, be aware of this option with use cases where it makes sense to connect to the AWS service directly with the SDK.



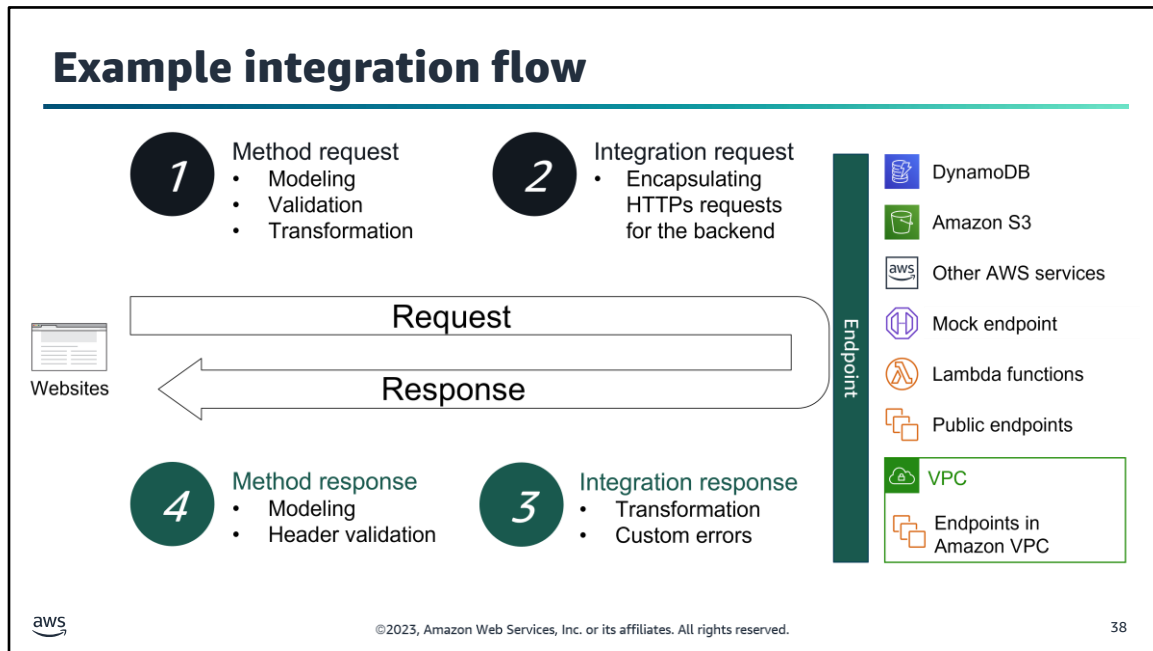
Regardless of the endpoint type that you choose for your API, you can integrate the API with Lambda and other AWS services.

Lambda and many other AWS services are not directly accessible by using HTTP URIs, so you cannot write standard HTTP calls to reach them directly. Instead, use API Gateway to proxy requests to AWS services on the backend.

API Gateway can run Lambda functions in your account, connect to DynamoDB or Amazon S3, and start AWS Step Functions state machines. API Gateway can also call HTTP endpoints that are hosted on AWS Elastic Beanstalk or Amazon Elastic Compute Cloud (Amazon EC2). In addition, it can call HTTP-based operations that are not AWS hosted but are accessible via the public internet. You can also integrate API Gateway with other AWS services directly. For example, you could expose an API method in API Gateway that sends data directly to Kinesis.

With REST APIs, you have the option of a mock endpoint. Mocks can be used to abstract options, but they are also useful for prototyping front-end applications. Your front end can call a reliable endpoint to ensure that all infrastructure is in place, and the mock endpoint can return default data. Then the mock can be swapped out with a real backend when you are ready to integrate it. A mock endpoint is also useful for delivering secure index.html pages. You'll learn more about mocks when you discuss cross-origin resource sharing (CORS).

With both REST and HTTP APIs, you can connect to a VPC link that allows access to resources in a VPC through an Elastic Load Balancer.

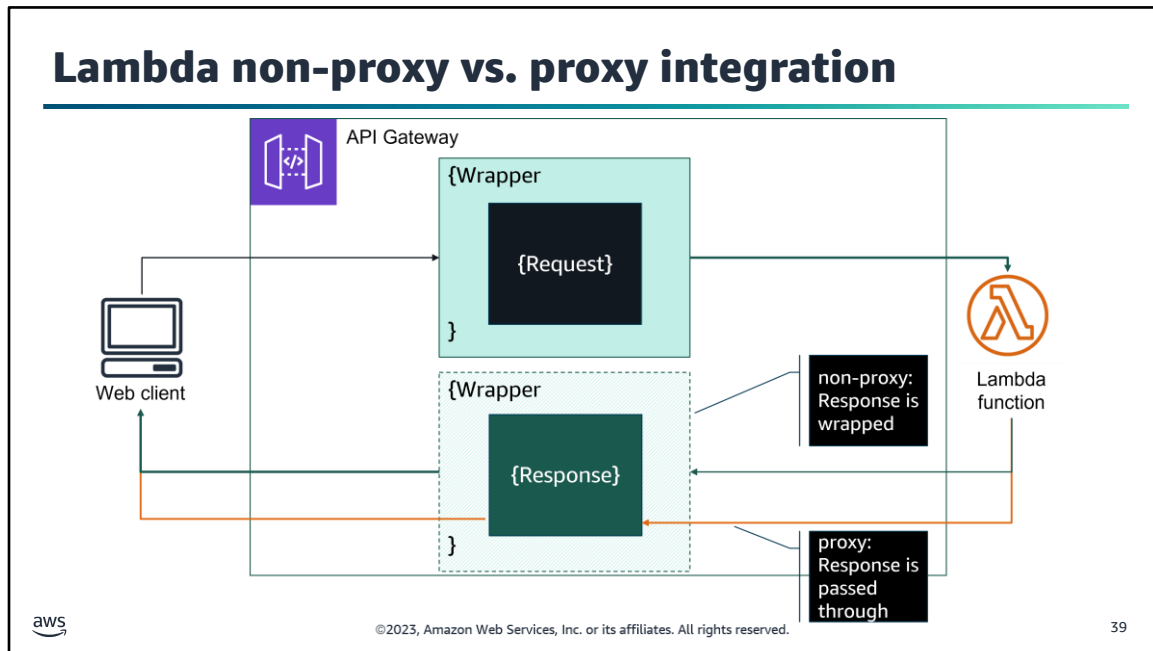


An API integration has an integration request and an integration response. An integration request encapsulates an HTTP request that the backend receives. It might or might not differ from the method request that the client submits. An integration response is an HTTP response that encapsulates the output that is returned from the backend.

A client uses the API to access a backend feature through the method request. If necessary, API Gateway translates the client request. It translates the request into the form that is acceptable to the backend in the integration request before forwarding the incoming request to the backend. The transformed request is known as the integration request. Similarly, the backend returns the response to API Gateway in the integration response. API Gateway then routes the integration response to the method response before it sends the response to the client. Again, if necessary, API Gateway can map the backend response data to a form that the client expects.

For more information about how to set up:

- REST API methods, see “Set Up REST API Methods in API Gateway” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-method-settings.html>.
- REST API integrations, see “Setting Up REST API Integrations” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-integration-settings.html>.



API Gateway wraps client requests with metadata that Lambda needs and passes the wrapped request to the target function. The response is handled differently depending on whether you choose a non-proxy (custom) or proxy integration between API Gateway and Lambda.

Suppose that you choose a non-proxy (custom) integration between API Gateway and Lambda. When the Lambda function responds, API Gateway wraps the response to make it a valid response for a browser or other HTTP client. If API successfully gets a response from the function, it includes a status code 200 to indicate that the call to the function was successful.

When you choose proxy as the API Gateway integration type for a Lambda function, requests are still wrapped with the metadata that Lambda needs. However, responses are passed through API Gateway without any wrapper. Your Lambda function response must be in a format that the web browser or HTTP client understands. It must follow the JSON output format that is provided in the developer guide (For more information, see “Output format of a Lambda function for proxy integration” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/set-up-lambda-proxy-integrations.html>).

Suppose that you have an API that only needs to forward HTTP requests to a Lambda function that can respond to the client. Setting up a proxy integration is simpler than setting up a direct integration, but your Lambda function must handle creation of an appropriate response.

Use non-proxy (custom) integrations to have API Gateway wrap the response for use by an HTTP client. You can also use custom integrations when you must transform the Lambda response in some way before returning it to the client.

Example standard response in non-proxy integration

Lambda function response

```
{ "message": "Hello from Lambda!" }
```

API Gateway standard response in non-proxy integration

```
{
  "isBase64Encoded": false,
  "statusCode": 200,
  "body": "{ \"message\": \"Hello from Lambda!\" }",
  "headers": {
    "content-type": "application/json"
  }
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

40

This example shows a non-proxy (custom) integration between API Gateway and a Lambda function. In the example, a JSON response "Hello from Lambda" for your Lambda function is set up by using the Lambda console to test your function.

API Gateway automatically wraps that message to make it a valid response for a browser (or client that understands HTTP). It uses the structure that is illustrated on the slide. The response includes the 200 status code to indicate a successful response.

Example Lambda error response in proxy integration

Lambda
permissions error
message

```
{ "message": "You are not allowed to see  
this content" }
```

Lambda
customized
response in a
proxy integration

```
{  
  "cookies" : ["cookie1", "cookie2"],  
  "isBase64Encoded": true,  
  "statusCode": 403,  
  "headers": { "headername": "headervalue", ... },  
  "body": "You are not allowed to see this content"  
}
```



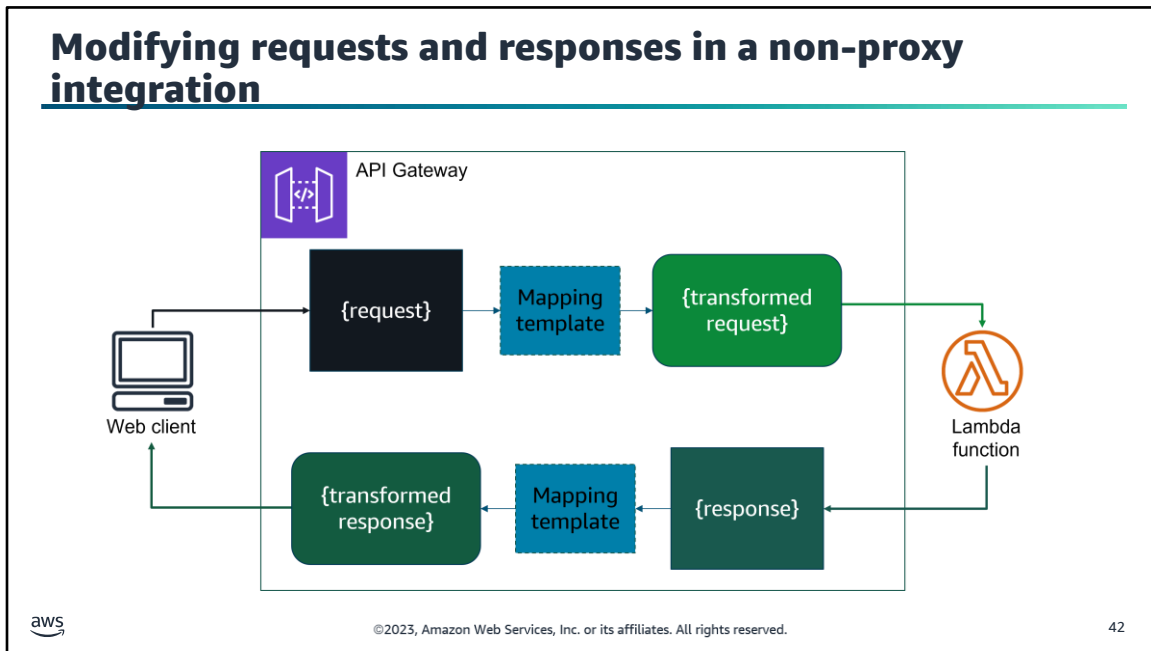
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

41

Sometimes, even though the call is successful, you might want to pass error information back to the client, rather than returning a 200 OK status. In this example, your Lambda function returns a 403 error with the message that the request has a permissions issue. This call to the API is successful, so API Gateway would treat the call as a 200 OK response. However, what is important to your client is that the request has a permissions issue.

To handle function errors that you want to return to the client with a non-proxy integration, map the response errors to standard HTTP error responses. In this way, your clients don't receive the response as a 200 status.

With a Lambda proxy integration, you use the output format that API Gateway requires. Include the error information as part of your customized Lambda response as illustrated on the slide.



When you choose a non-proxy (custom) integration with Lambda, API Gateway can modify or enrich the request or the response rather than merely wrapping it. You can use API Gateway mapping templates, which are written in Apache Velocity Template Language (VTL), to transform requests. For example, you might use mapping to delete, add, or edit parameters.

By using this method, you don't need to write your Lambda response so that a browser understands it. You also gain more control over the response that is returned to the client. You can let the Lambda function return JSON. You can use mapping template VTL to modify the response to include the formatting and status codes that the client needs. You can also use mapping templates to handle custom errors.

Example: Modifying a request by using VTL

Before transformation

```
{
  "firstname": "John",
  "lastname": "Stiles",
  "phone": "555-555-5555"
  "city": "Miami",
  "state": "Florida",
  "favoritePizza": "pineapple"
}
```

After transformation

```
{
  "name": "John Stiles",
  "phone": "555-555-5555",
  "address": {
    "city": "Miami",
    "state": "Florida"
  }
  "favoritePizza": "Hawaiian"
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

43

In this example, you use VTL to map some of the data on the incoming request to the format that the backend requires. For example, the first and last name are combined into name, the city and state become part of an address element, and the favorite pizza is changed from pineapple to Hawaiian.

Transforming the response

Before VTL transformation

```
{  
  "message": "Hello from Lambda",  
}
```

After VTL transformation

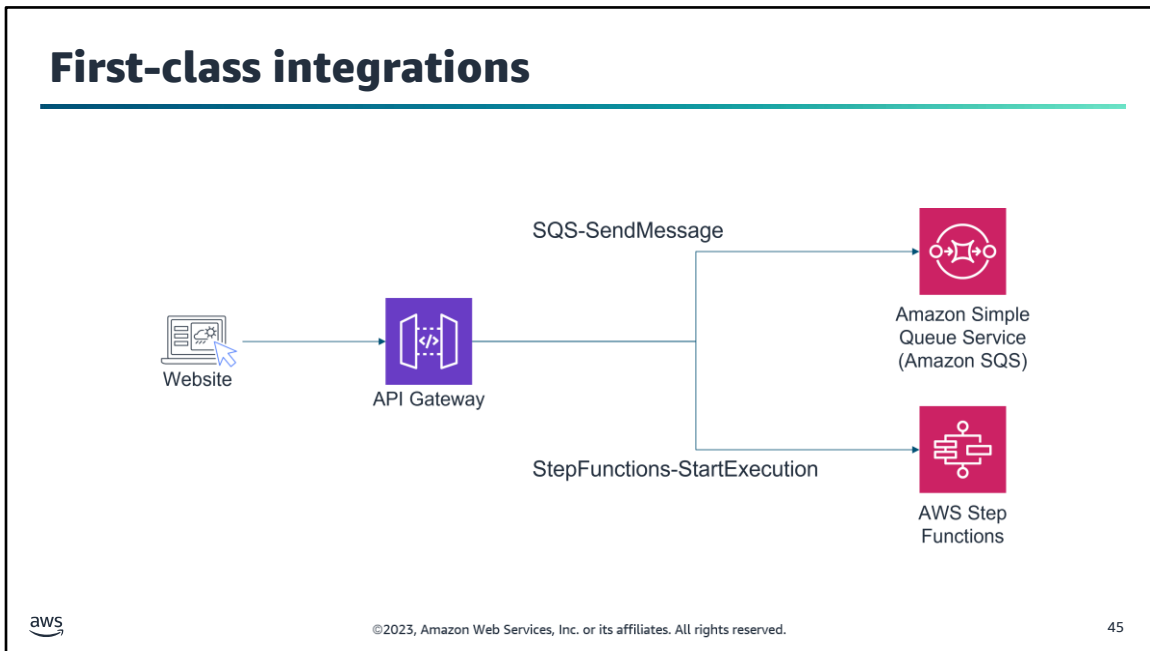
```
{  
  "cookies": [...],  
  "isBase64Encoded": false,  
  "statusCode": 200,  
  "headers": [... ],  
  "body": "Hello from Lambda"  
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

44

In this example, the Lambda function returns the simple JSON message “Hello from Lambda”, and the mapping template transforms it for use by a browser.



A first-class integration connects an HTTP API route to an AWS service API, which is similar to how API Gateway integrates with Lambda functions.

With first-class integrations, when a request hits API Gateway, it will invoke the specified AWS service API for you.

You can use first-class integrations to send a message to an Amazon Simple Queue Service (Amazon SQS) queue, or start a Step Functions state machine. The API Gateway developer guide includes a list of available first-class integrations for HTTP APIs on the Integration subtyped reference page at <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-develop-integrations-aws-services-reference.html>.

Each service requires a different mapping. You still must provide the parameters that each service needs. For example, Amazon Simple Notification Service (Amazon SNS) requires a different object with keys and values than a call to DynamoDB.

API Gateway must map the parameters in the BODY, QUERYSTRING, or HEADER value of the incoming request to an object that the target AWS service needs.

Example: First-class integration

```
aws apigatewayv2 create-integration \  
  --api-id abcdef123 \  
  --integration-subtype SQS-SendMessage \  
  --integration-type AWS_PROXY \  
  --payload-format-version 1.0 \  
  --credentials-arn arn:aws:iam::123456789012:role/apigateway-  
sqs \  
  --request-parameters '{"QueueUrl":  
"$request.header.queueUrl", "MessageBody":  
"$request.body.message"}'
```

Connect an HTTP API
route to an AWS
service API

You provide the
mapping that the
service requires

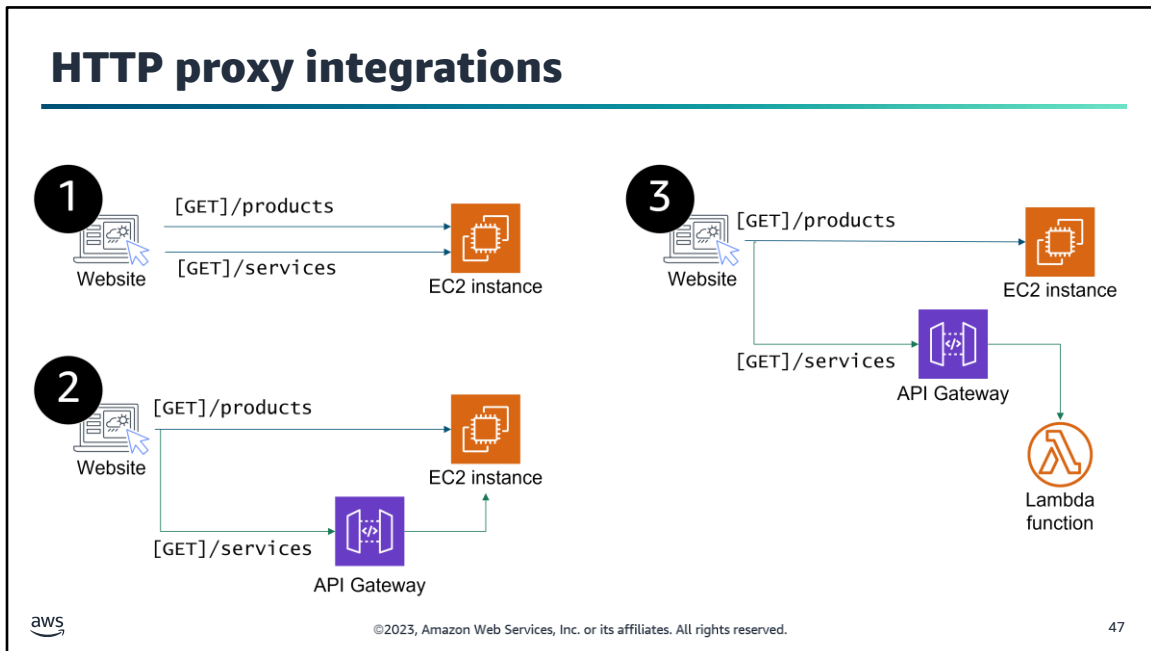


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

46

This Amazon SQS example forwards the QueueUrl and message value in the request's body to Amazon SQS to process.

When you use the AWS console to work with an AWS service, it cannot know which mappings your application is using. Therefore, you must set up the --request-parameters.

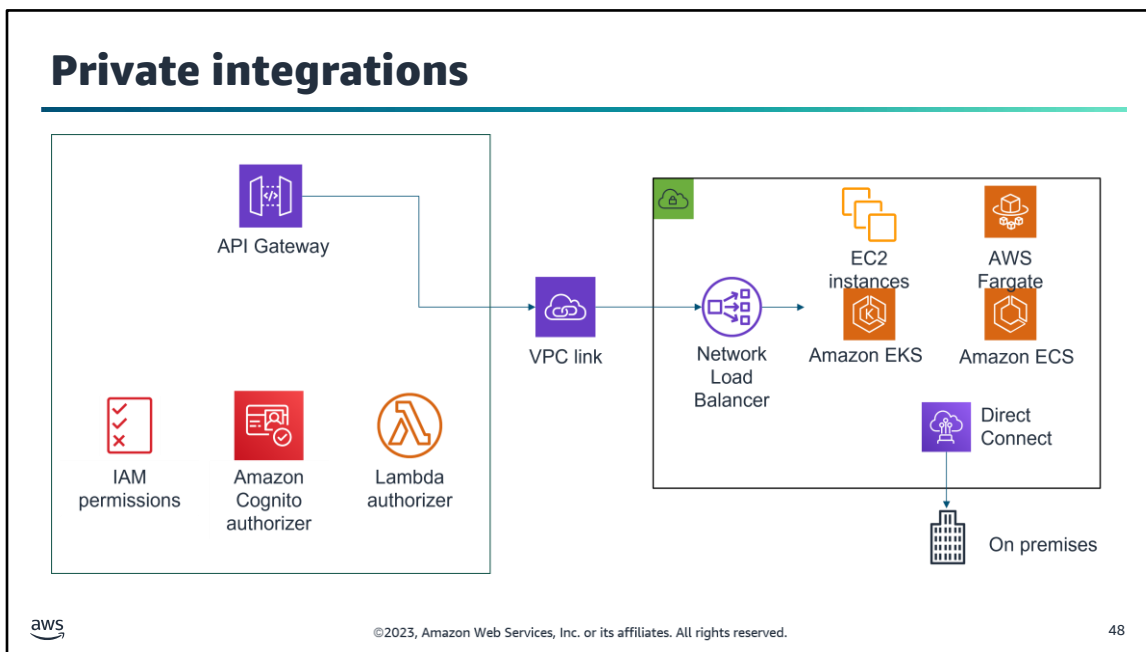


You can use HTTP proxy integrations to direct a route to a resource on the internet. This method is useful when migrating from a monolithic application to web services.

You can map your entire monolith with API Gateway routes and forward them to your monolith's endpoints. Then, one by one, you can remap specific routes to point to your new Lambda function services as you break off pieces of the monolith.

In this example,

1. You start with two API routes that connect to your monolithic application on Amazon EC2.
2. Point the `/services` route to API Gateway and use an HTTP proxy integration to point it back to your monolith endpoint on Amazon EC2.
3. When your updated version of the `/services` service is written in a Lambda function, you point the API integration to the Lambda function instead.



API Gateway can access resources within a VPC by using a VPC link with Application Load Balancer or Network Load Balancer. It can also use a VPC link with resources that are registered with an AWS Cloud Map service inside your account's VPC.

Example: Creating a private integration

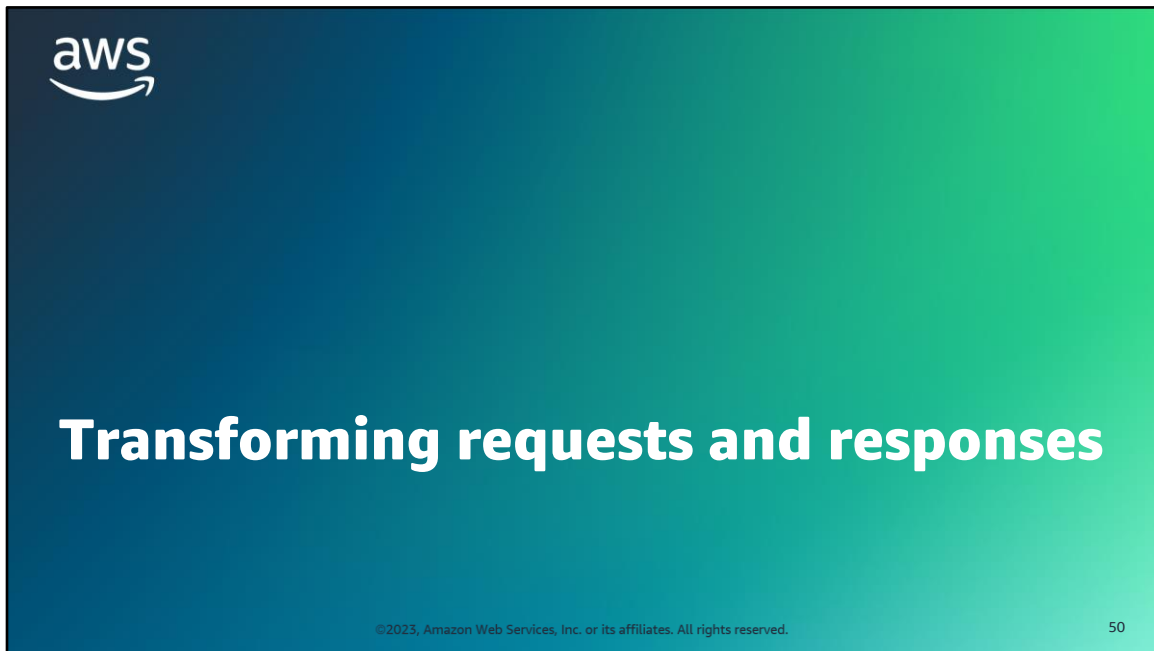
```
aws apigatewayv2 create-integration \  
    --api-id api-id \  
    --integration-type HTTP_PROXY \  
    --integration-method GET \  
    --connection-type VPC_LINK \  
    --connection-id VPC-link-ID \  
    --integration-uri arn:aws:elasticloadbalancing:us-east-2:123456789012:listener/app/my-load-balancer/50dc6c495c0c9188/0467ef3c8400ae65 \  
    --payload-format-version 1.0
```



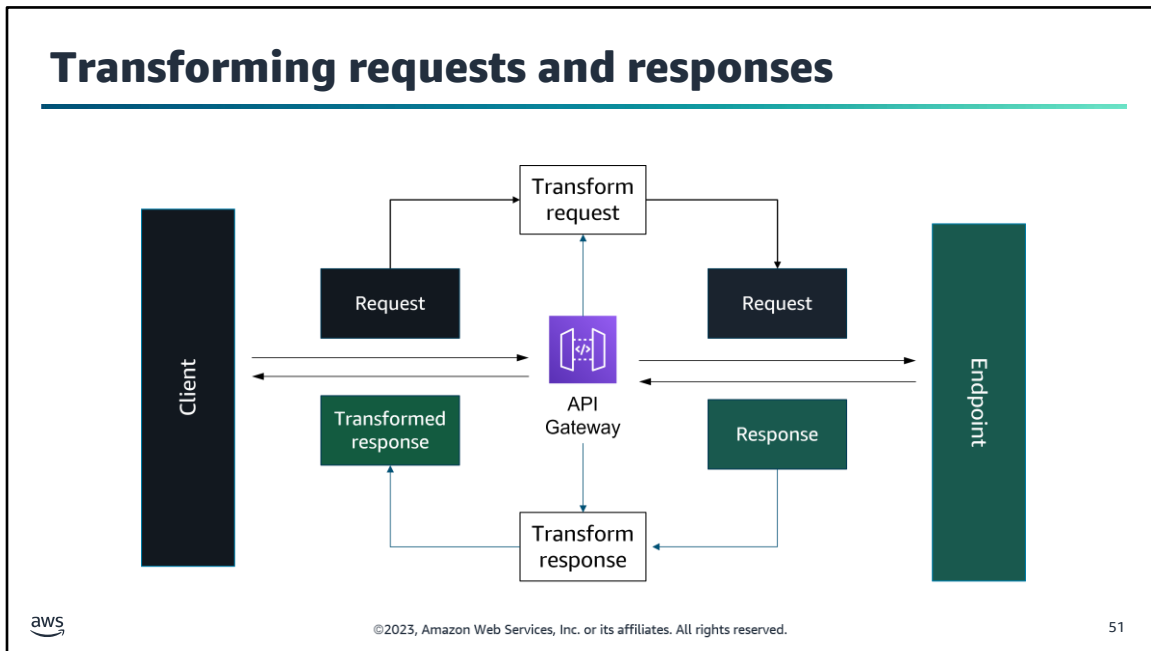
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

49

This example assumes that you have created your VPC link. Then, you use the create-integration method to connect API Gateway to VPC resources by using the VPC link.



Transforming requests and responses



As illustrated in the Lambda non-proxy integration example, you can modify API requests from clients before they reach your backend integrations. You can also change the response from integrations before API Gateway returns the response to clients. You use *parameter mapping* to modify API requests and responses for HTTP APIs. To use parameter mapping, you specify API request or response parameters to modify, and then specify how to modify those parameters.

Notice that you cannot configure request or response mappings for reserved headers. For information about reserved headers, see the Amazon API Gateway Developer Guide section “Transforming API Requests and Responses” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-parameter-mapping.html>.

Example request transformation

```
aws apigatewayv2 create-integration \  
  --api-id abcdef123 \  
  --integration-type HTTP_PROXY \  
  --payload-format-version 1.0 \  
  --integration-uri 'https://api.example.com' \  
  --integration-method ANY \  
  --request-parameters '{ "append:header.header1": "$context.requestId" }'
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

52

You can append, overwrite, or remove any header value that you choose.

You can also do these tasks for query string values, and you can override entire paths.

For example, to add a custom header to the request before it arrives at your target, use something like the example on the slide.

Section 5 key takeaways



- API Gateway simplifies the work to integrate with AWS services from your client applications.
- Use Lambda proxy and HTTP proxy integrations for simple passthroughs to backend services.
- Use transformation options to modify requests and responses between clients and targets.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

53

Some key takeaways from this section of the module include:

- API Gateway simplifies the work to integrate with AWS services from your client applications.
- Use Lambda proxy and HTTP proxy integrations for simple passthroughs to backend services.
- Use transformation options to modify requests and responses between clients and targets.



Section 6: Deploying an API.

Deployments are associated to a stage

- Stages are snapshots of an API:
 - Named as you like
 - Identified by API ID and stage name
 - Included in the URL that is used to invoke the API
- Developers create multiple stages to:
 - Differentiate versions for development environments (like dev and prod)
 - Connect different stages to different backends by using stage variables
 - Optimize a particular deployment—for example, enabling caching or throttling per stage



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

55

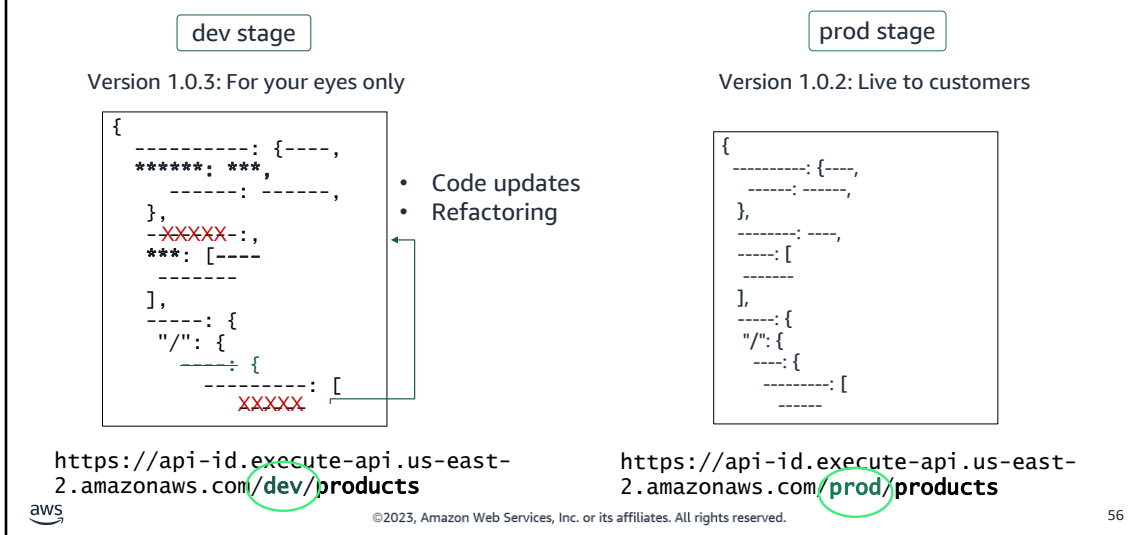
After you create your API, you must deploy it to make it callable by your users. To deploy an API, you create an API deployment and associate it with a *stage*. A stage must be selected when deploying a REST or WebSocket API. The HTTP API deploys to a default stage when you create it to make it simpler for you to deploy. You can set any stage in an HTTP API to autodeploy.

You can use a stage to manage and optimize a particular deployment. For example, you can set up stage settings to enable caching, customize request throttling, or configure logging. You can use stage variables to connect different API stages to different versions of a backend (for example, two different Lambda function versions).

For information about how to set up a stage in API Gateway, see the API Gateway Developer Guide:

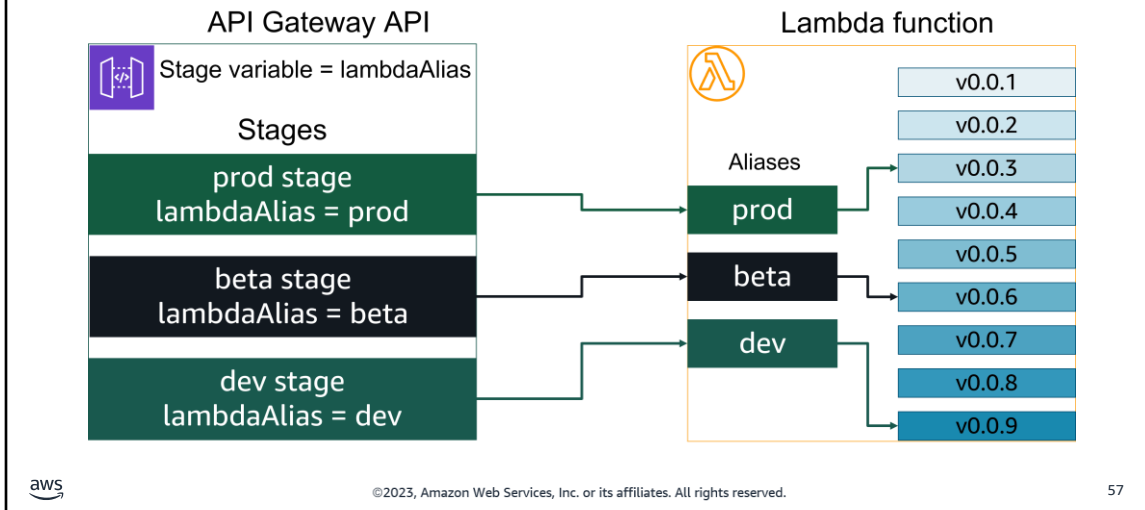
- “Setting Up A Stage for REST API” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/set-up-stages.html>
- “Setting Up Stage Variables for a REST API Deployment” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/stage-variables.html>
- “Deploy a WebSocket API in API Gateway” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-set-up-websocket-deployment.html>

Using stages as development environments



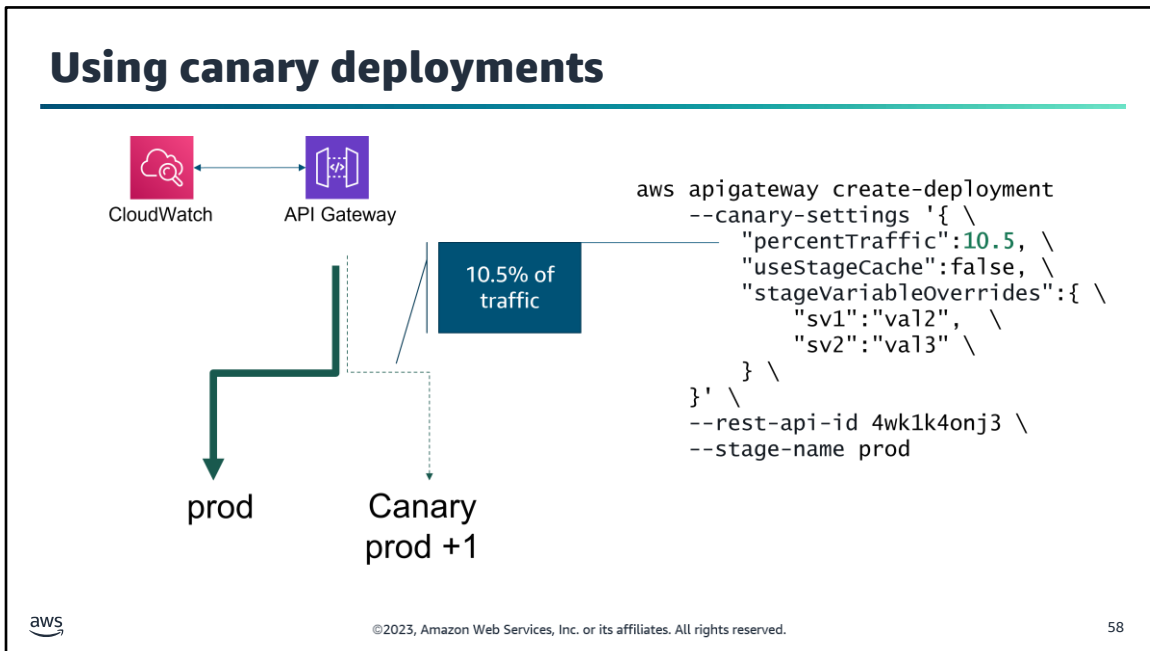
As was mentioned earlier, one use case for stages is to differentiate API versions between development and production. Using a dynamic AJAX-based website as the example client, you could deploy the API in a production stage connected to a production Lambda function. Then, continue to improve the code in another version of the Lambda function and test it by using the dev stage of the API.

Connecting to different backends by using stage variables



With stage-level variables, you can set variables that are different per stage. This technique helps developers target different resources (for example, pointing to a development database rather than the production database) without writing different code.

Continuing the dynamic website example, you can use API Gateway stage variables to programmatically tie different API versions to different versions of your Lambda functions. You can use Lambda aliases as the stage variable values. You will learn more about Lambda aliases in Module 7.



Canary release is a software development strategy in which a new version of an API (or other software) is deployed for testing purposes. The base version remains deployed as a production release for normal operations on the same stage. In a canary release deployment, total API traffic is separated at random into a production release and a canary release with a preconfigured ratio.

To deploy an API with a canary release, you create a canary release deployment by adding canary settings to the stage of a regular deployment. The canary settings describe the underlying canary release, and the stage represents the production release of the API within this deployment. API Gateway handles sending the traffic to the canary on your specified stage.

This strategy is a good way to catch issues and minimize any impact to most of the API users.

The code excerpt on this slide is part of a deployment that uses a canary to send 10.5% of traffic to the canary.

Updating canary deployments



```
aws apigateway update-stage \
  --rest-api-id 4wk1k4onj3 \
  --stage-name prod \
  --patch-operations \
  op=replace,path=/canarySettings/percentTraffic, \
  value=25.0
```

prod +1



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

59

If the results of the canary testing are successful, increase the percentage of traffic that goes to the canary. Continue to monitor the result to ensure that nothing unexpected happens.

You can continue updating the percentage until you are ready to promote the canary version to the production version.

When you promote the canary, the stage is fully deployed to production.

Section 6 key takeaways



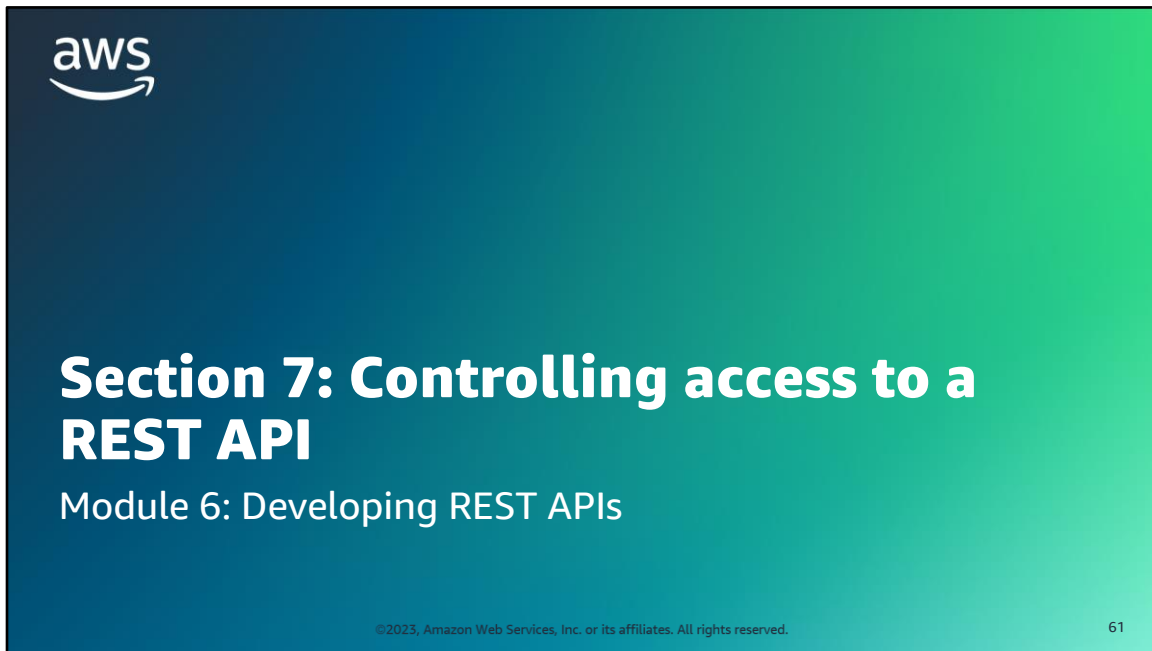
- Stages are snapshots of an API version and are required to deploy your API.
- You can use stages to differentiate deployments for different development environments or backends.
- Use canary deployments to test out an API by sending a percentage of traffic to a new version.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

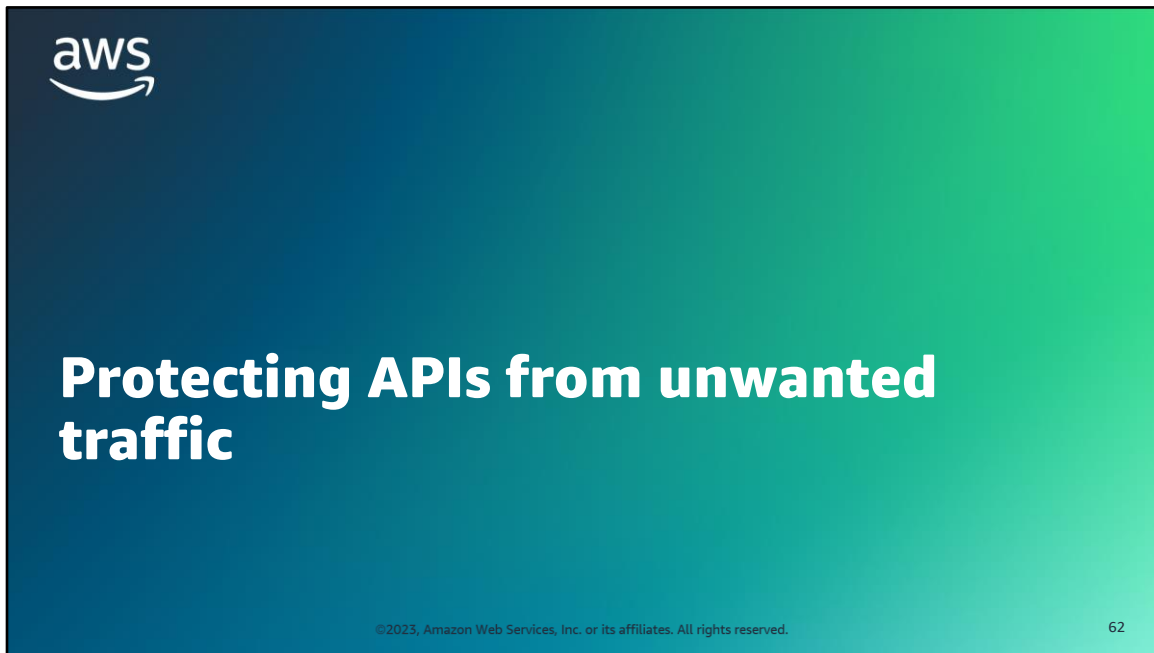
60

Some key takeaways from this section of the module include:

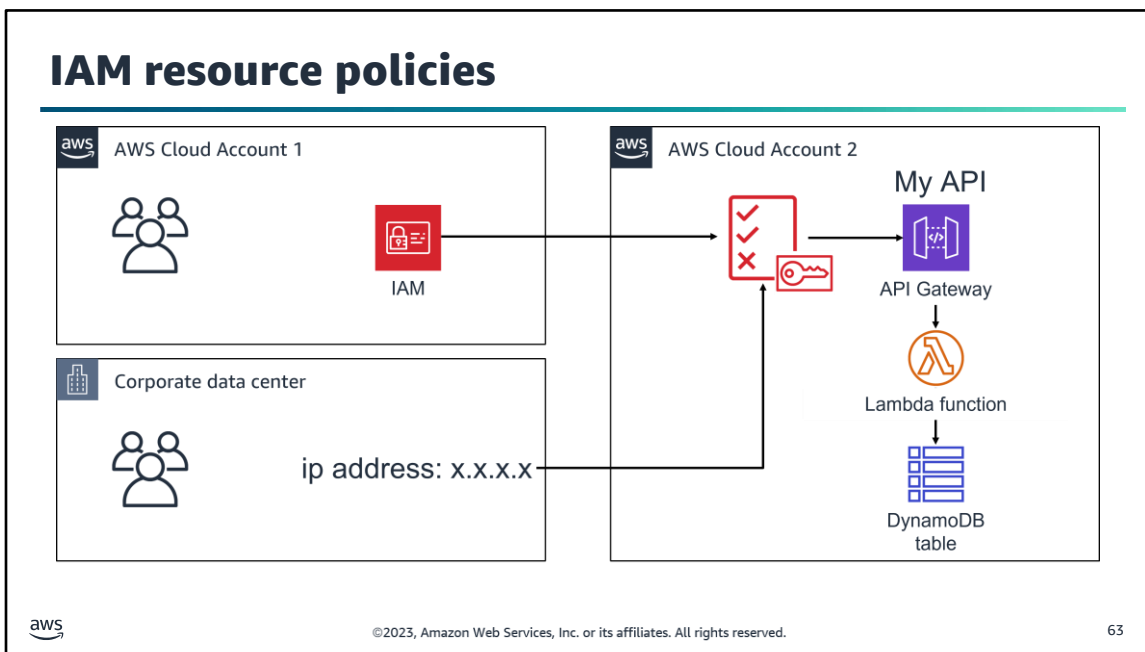
- Stages are snapshots of an API version and are required to deploy your API.
- You can use stages to differentiate deployments for different development environments or backends.
- You can use canary deployments to test out an API by sending a percentage of traffic to a new version.



Section 7: Controlling access to a REST API.



Protecting APIs from unwanted traffic



With REST APIs, you can use AWS Identity and Access Management (IAM) resource policies to control access to your API's endpoint without using other services like AWS WAF. For example, you might use IAM permissions to allow users in a different AWS account to access your API. Alternatively, you might limit access to your API's dev stage to only people in your office (by using the IP address of the office network).

HTTP APIs currently cannot use resource policies in this way.

Custom domains for certificates

`https://12345.execute-api.us-east-1.amazonaws.com/prod`

`https://*.mydomain.com/api-one`

Secure Sockets Layer
(SSL) certificates
managed through AWS
Certificate
Manager (ACM)

Supports wildcard
(*) domains

Supports multiple
domains through base
path mapping



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

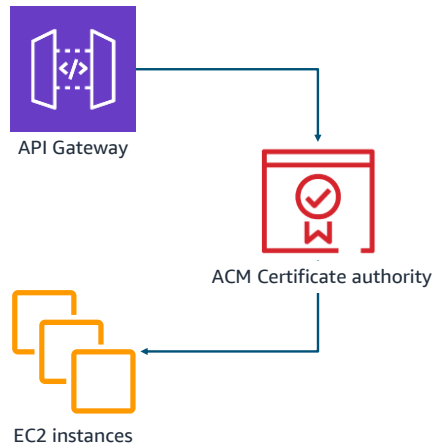
64

To support the use of certificates as a method of controlling access to your APIs, you first set up a custom domain for your API.

Setting up custom domains for API Gateway and setting up the required ROUTE 53 entries is beyond the scope of this class. For more information, see the API Gateway Developer Guide:

- [Setting Up Custom Domain Names for REST APIs](#)
- [Setting Up Custom Domain Names for HTTP APIs](#)
- [Setting Up Custom Domain Names for WebSocket APIs](#)

Client certificates



- Generate a client-side SSL certificate by using API Gateway
- The backend verifies the request that comes from API Gateway by using a public key
- Expires after 365 days



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

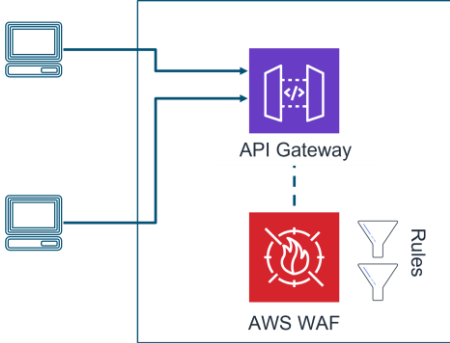
65

After you have your custom domain set up, you can set up your certificate. When the certificate is set up, your EC2 instances can confirm that a request came in from API Gateway.

If you have devices such as Internet of Things (IoT) devices, you can install certificates on these devices. Then, API Gateway can ensure that they are valid when requests come in.

The process uses Secure Sockets Layer (SSL) or more specifically, Transport Layer Security (TLS). Even more specifically, it would be mutual TLS because both API Gateway and the IoT device have a certificate. This mutual TLS is possible only when you use a custom domain with your API Gateway. In that way, your API has a certificate that your IoT device can confirm.

AWS WAF



- Protect APIs from common web exploits like SQL injection and cross-site scripting (XSS)
- Matches a specified string or regular expression patterns
- Blocks requests based on origination from:
 - Specified IP address ranges
 - A specific country or region
 - Specific user agents, bad bots, and content scrapers

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

66

AWS WAF is a web application firewall that helps protect your web applications or APIs against common web exploits. You can deploy AWS WAF on CloudFront as part of your content delivery network (CDN) solution or on API Gateway for your APIs. You can use managed rules, which are preconfigured sets of rules. The managed rules for AWS WAF address common security risks. These rules are regularly updated.

With AWS WAF, you can:

- Protect APIs from common web exploits like SQL injection and cross-site scripting (XSS)
- Match specified string or regular expression patterns in HTTP headers, methods, query strings, URIs, and the request body
- Block requests based on their origin, from:
 - Specified IP address ranges
 - A specific country or region
 - Specific user agents, bad bots, and content scrapers

For more information, see “Using AWS WAF to protect your APIs” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-aws-waf.html>.

CORS prevents cross-origin requests



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

67

Cross-origin resource sharing (CORS) is a browser security feature that restricts cross-origin HTTP requests that are initiated from scripts that run in the browser. The browser enforces CORS, which is relevant to you only if your clients are browsers. If your API will receive cross-origin requests, you should enable CORS support. With CORS support, you can accept requests that you want while preventing cross-origin requests to domains that don't explicitly allow the originating domain or origin.

A *cross-origin* HTTP request is a request that is made to:

- A different *domain* (for example, from example.com to amazondomains.com)
- A different *subdomain* (for example, from example.com to petstore.example.com)
- A different *port* (for example, from example.com to example.com:10777)
- A different *protocol* (for example, from https://example.com to http://example.com)

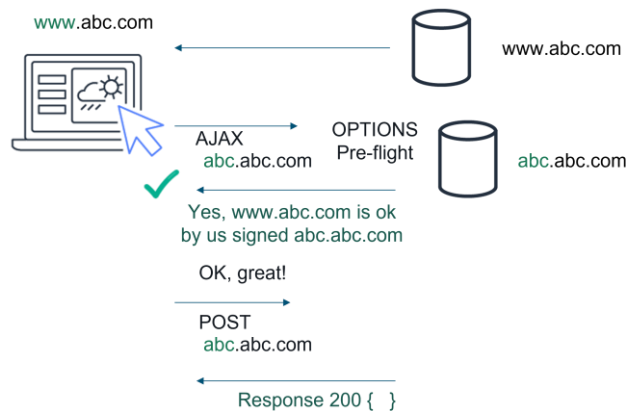
Web-based applications that are dynamic and thus make AJAX requests to servers will likely run into CORS at some point.

For example, suppose that a web application is served from an Amazon S3 bucket. You have API Gateway running in front of a Lambda function to get products from DynamoDB. Then, the browser will block the request to DynamoDB as a cross-origin request.

The browser knows which domain it is on — for example, “abc.example.com”. When it makes AJAX requests to anything in that same domain such as “abc.example.com/products”, CORS support is not required.

However, your web application might make a “non-simple” AJAX request that is cross domain, such as xyz.example.com or abc.some_third_party.com. In this case, the browser thinks that you are at one site and another unknown, untrusted site is trying to inject data into your webpage. As a result, the browser prevents the action.

Cross-origin requests with CORS support



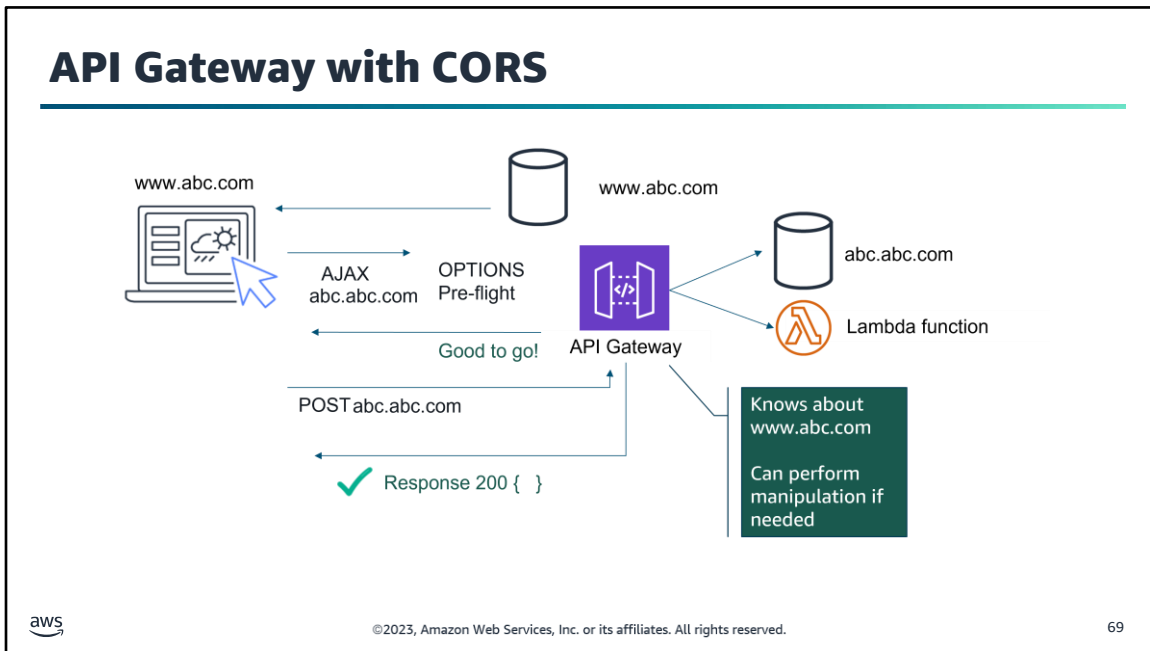
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

68

To allow these types of requests, CORS requires a preflight request that uses the options method to find out which methods are allowed. If the type of request that is specified in the preflight request is approved, then the actual request gets sent. That call has a *Time to Live* (TTL) setting, which you configure. When you enable CORS for your API, you specify a list of allowed headers, methods, and origins that might access the resource. They are used in response to the preflight request. Requests that do not meet the criteria for the allowed headers, methods, and origins will generate an error.

Notice that simple AJAX requests such as GETs are generally read-only (that is, they are not writing anything to a server or database). They are considered *simple* requests that do not require CORS support.

Sites that are frequently consumed, such as Twitter, now require API dev keys via POST. Thus, they are not simple requests, so the browser will use CORS protection for any requests. At one time, these APIs allowed GET requests from anyone, but they no longer do so.



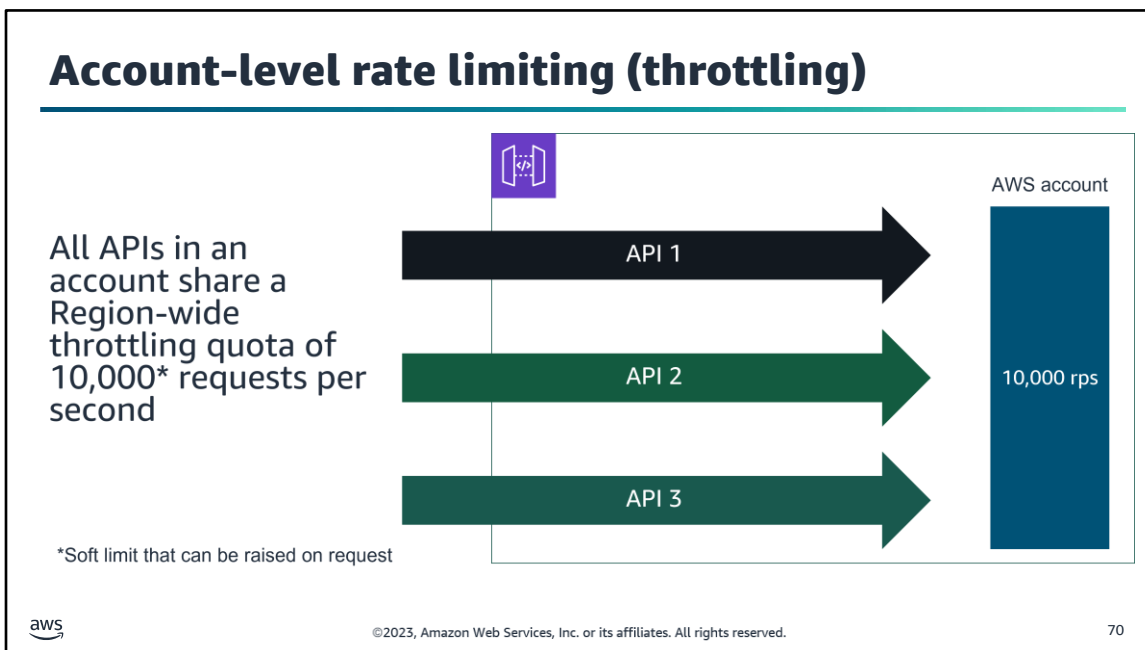
API Gateway can be set up as a proxy to handle all the preflight request options work. Therefore, you don't need to be concerned about managing that part at the server. Using API Gateway with Lambda means that you have less code to write to be able to handle CORS inside your Lambda code.

You can configure CORS headers on API Gateway to direct API clients to invoke API calls only from allowed origins. You can enable and configure CORS on both REST and HTTP APIs.

When you use a REST API and configure CORS, API Gateway sets up a mock endpoint to handle the options method. The setup is simplified when you use HTTP APIs.

For more information, see:

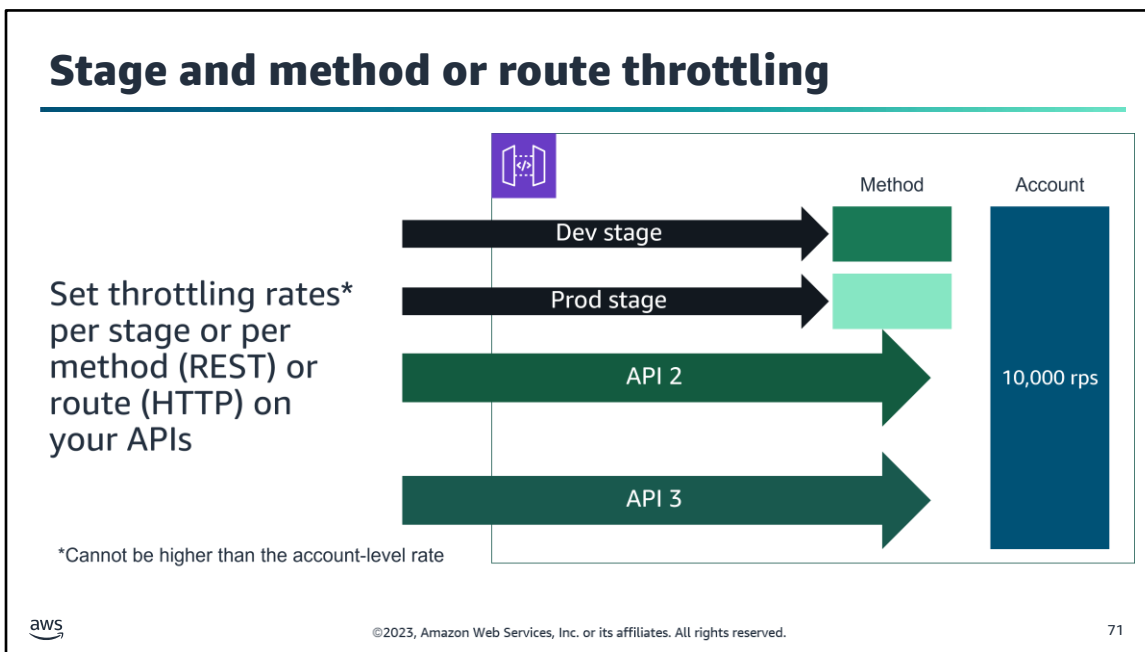
- “Enabling CORS for a REST API Resource” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-cors.html>
- “Configuring CORS for an HTTP API” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-cors.html>
- “Testing CORS” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-test-cors.html>



Throttling helps to protect your APIs from rate-based attacks, and API Gateway APIs share a Region-wide quota of 10,000 requests per second.

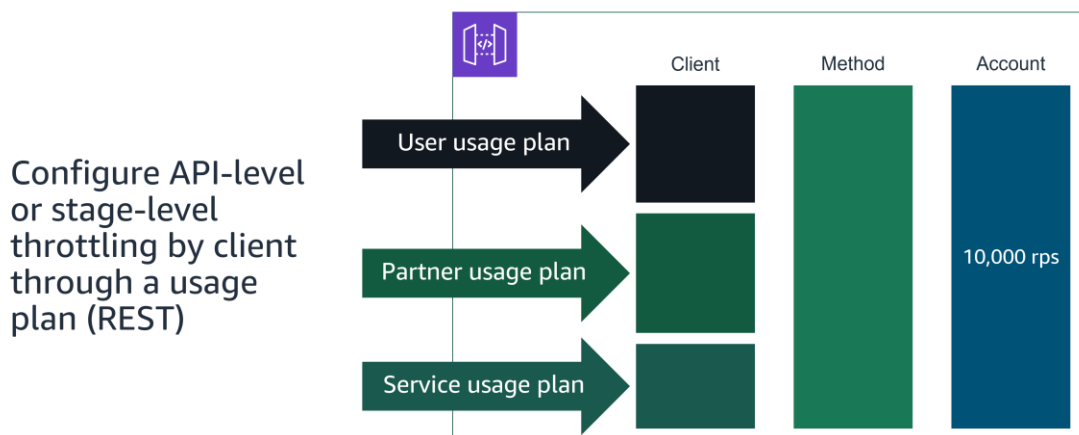
You can also set quotas per API, on individual methods, and by client to manage the volume of requests that hit your APIs.

If you receive a “429 too many requests” error in response to an API Gateway request, it is an indication that the request was throttled.



You can set throttling rates per stage or per method (for REST APIs) or routes (for HTTP APIs).

Throttling by client with a usage plan



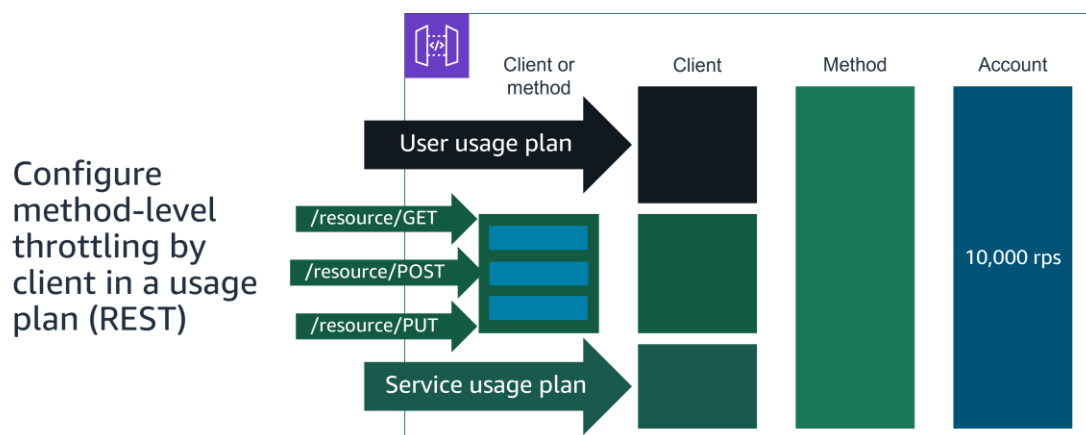
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

72

A usage plan specifies who can access deployed stages and methods, and controls the rate and number of requests that a client makes. Within usage plans, you can set throttling limits to control the request rate. You also can set a quota to control how many requests an API key can use within a specific time frame. API keys (REST) are unique string values that you give out to grant access to APIs.

For information about creating and using usage plans with API keys, see “Creating and Using Usage Plans with API Keys” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-usage-plans.html>

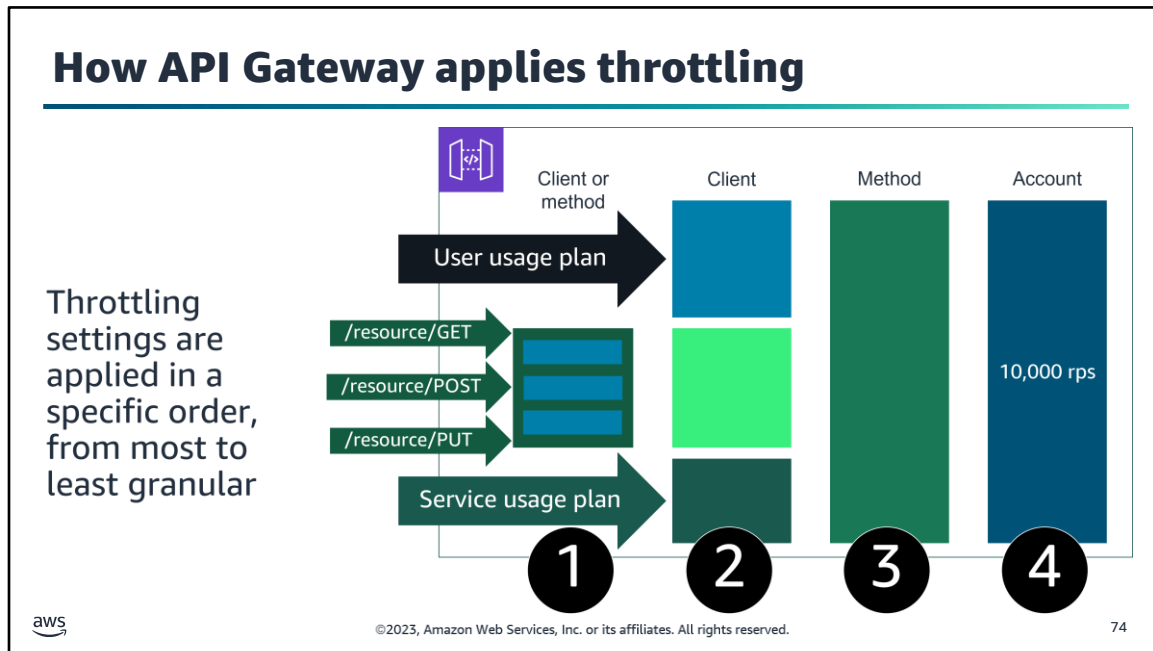
Throttling by client and method



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

73

With REST APIs, you can associate throttling by method to specific clients in a usage plan.

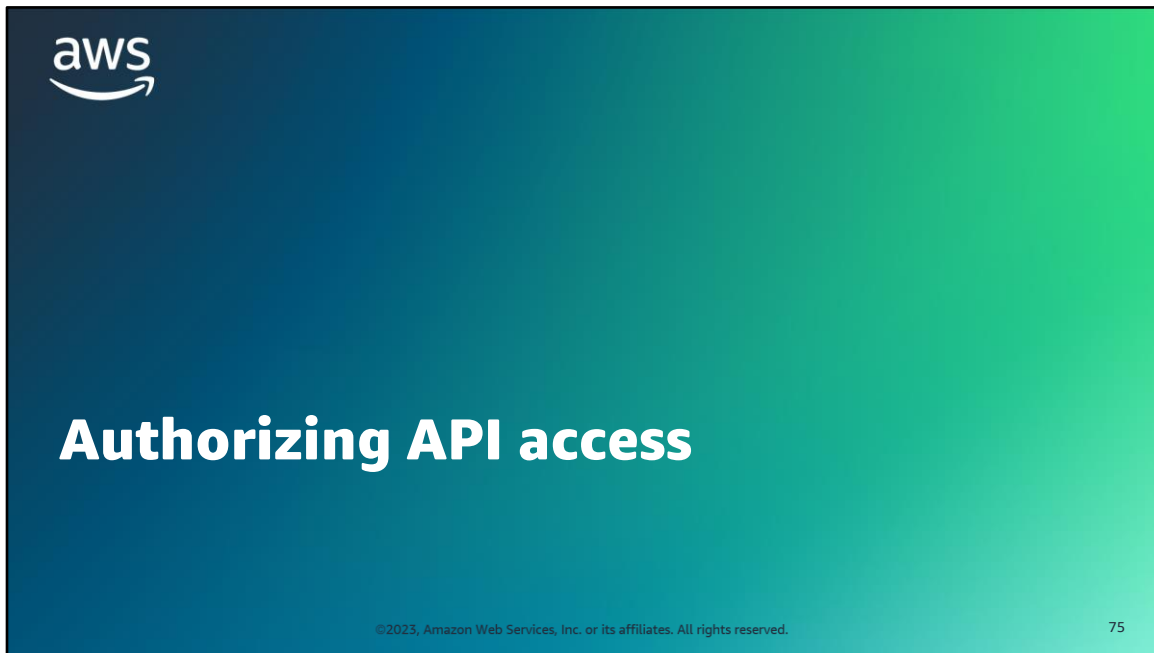


Settings are applied first to the most granular control. For example, suppose that an incoming request is from a client that is tied to a usage plan on a REST API. Throttling that is related to individual methods for that client is applied first and then followed by any client-level limits. Next, any throttling settings that are indicated for a particular method and stage are applied, and followed by any limits placed across the entire stage. Finally, if no throttling is applied at more granular levels, the account quota is applied.

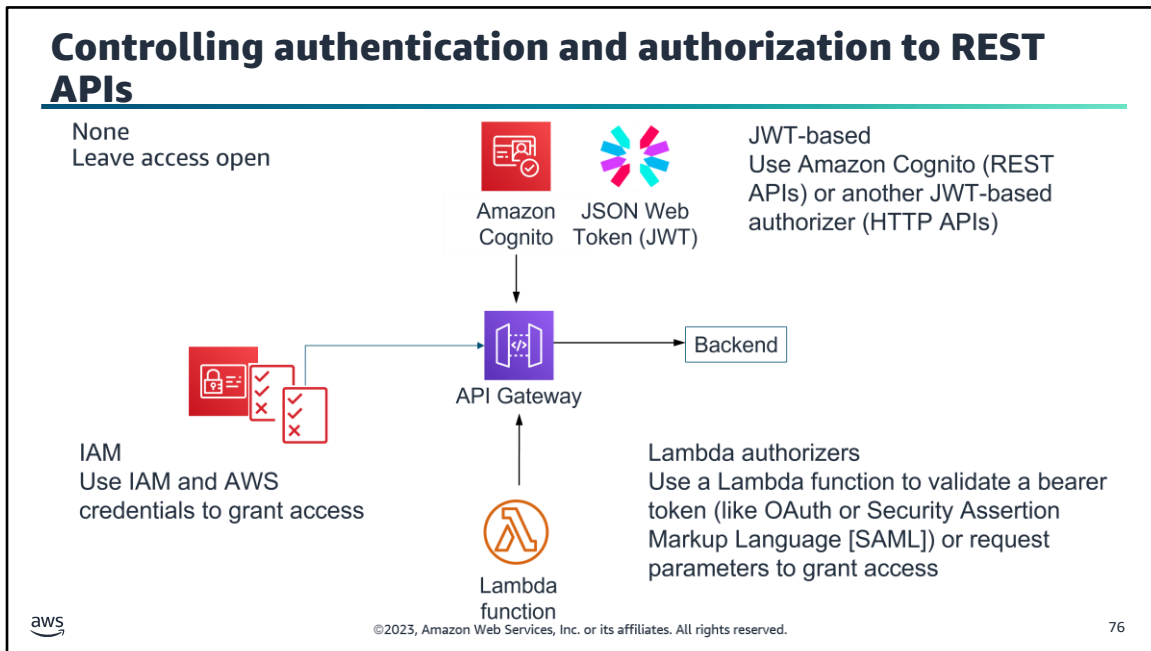
For more information about throttling requests to your HTTP API, see “Throttling requests to your HTTP” at

<https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-throttling.html>. For more information about throttling API requests for better throughput, see “Throttle API requests for better throughput” at

<https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-request-throttling.html>.



Authorizing API access



You have the following options for authorizers in API Gateway:

- None (open)
- IAM
- JSON Web Token (JWT)-based, including Amazon Cognito
- Lambda

IAM

Setting your API to authorize by using IAM means that your client application is using IAM credentials. Usually, these credentials are temporary ones with the use of the client-side SDKs. All requests that are not signed requests (from the SDK) return a permissions error to your SDK.

JWT

JSON Web Tokens are used in the background when using OpenID Connect (OIDC) or OAuth 2.0 to authenticate requests. REST APIs can use Amazon Cognito as their JWT authorizer, and HTTP APIs can use third-party providers.

When a request includes a JWT token as part of the request, API Gateway evaluates the request. It determines the request's scope of use, and ultimately validates whether this user can access the resource that they are requesting.

This validation can also validate against users in an Amazon Cognito user pool. You can configure distinct authorizers for each route of an API, or use the same authorizer for multiple routes.

To create a JWT-authorized API, you must have an identity provider to use. Amazon Cognito is a simple identity provider that you can use.

Example: Updating an existing route to use Amazon Cognito user pools

1. Create the authorizer

```
aws apigatewayv2 create-authorizer \  
  --name authorizer-name \  
  --api-id api-id \  
  --authorizer-type JWT \  
  --identity-source '$request.header.Authorization' \  
  --jwt-configuration Audience=audience,Issuer=https://cognito-  
  idp.us-east-2.amazonaws.com/userPoolID
```

2. Update the route to use it

```
aws apigatewayv2 update-route \  
  --api-id api-id \  
  --route-id route-id \  
  --authorization-type JWT \  
  --authorizer-id authorizer-id \  
  --authorization-scopes user.id user.email
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

77

For example, you might want to update an existing route to allow for an Amazon Cognito authorization by using JWT. You would first create the authorizer and then update the API route to use it.

The code snippets on this slide illustrate how you would do this update by using the AWS CLI.

Lambda authorizers

Function payload
without authorization
information

```
{
  "context": {
    "examplekey": "examplevalue"
  }
}
```

Function payload with
authorization
information

```
{
  "isAuthorized": true/false,
  "context": {
    "examplekey": "examplevalue"
  }
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

78

With a Lambda authorizer, you first send the request to a Lambda authorizer function. The purpose is to validate that the request meets your custom criteria and passes the result to the end target, often another Lambda function.

To the end target function, this validation means that it receives an additional `isAuthorized` Boolean value, as illustrated in the example on this slide.

As a result, the code in your end target function can more gracefully handle a rejection. It is preferable to returning an abrupt 40X error from API Gateway to the client.

Writing custom code for your Lambda authorizers is not part of this course. However, the API Gateway developer guide has additional information, and examples are available on GitHub and through the Lambda console.

For more information, see the following pages:

- “Use API Gateway Lambda Authorizers” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-use-lambda-authorizer.html>
- “Custom Authorizer Blueprints for AWS Lambda” at <https://github.com/aws-labs/aws-apigateway-lambda-authorizer-blueprints>

Section 7 key takeaways



- API Gateway supports the following protections from unwanted traffic:
 - Resource policies
 - Client certificates
 - AWS WAF
 - CORS
 - Throttling
- API Gateway supports the following options for authorizing access to resources:
 - IAM
 - JWT-based authorizers
 - Lambda authorizers

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

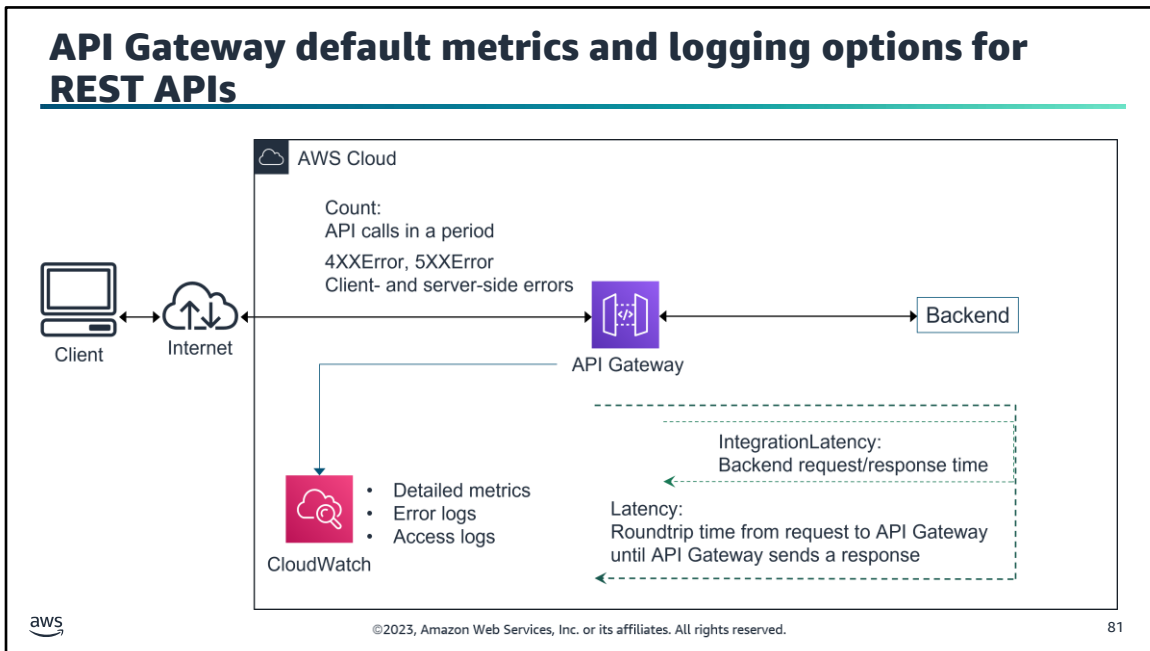
79

Some key takeaways from this section of the module include:

- API Gateway supports the following protections from unwanted traffic:
 - Resource policies
 - Client certificates
 - AWS WAF
 - CORS
 - Throttling
- API Gateway supports the following options for authorizing access to resources
 - IAM
 - JWT-based authorizers
 - Lambda authorizers



Section 8: Monitoring a REST API.



You can monitor APIs by using Amazon CloudWatch, which collects and processes raw data from API Gateway into readable, near-real-time metrics. These statistics are recorded for 15 months. Thus, you can access historical information and gain a better perspective on how your web application or service is performing.

By default, API Gateway sends the following metrics data to CloudWatch every minute:

- Count – Total number of API calls in a given period
- IntegrationLatency – Measures the responsiveness of the backend
- Latency – Time between when API Gateway receives a request from a client and when it returns a response to the client
- HTTP 400 and 500 errors – Number of client-side and server-side errors, respectively, that were captured in a given period

CloudWatch Logs provides optional logging of errors and access information for REST APIs that you configure by stage. For more information about how to configure logging, see “Setting Up CloudWatch Logging for a REST API” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/set-up-logging.html>.

For more information about how to monitor APIs with CloudWatch, see “Monitoring REST API Execution with Amazon CloudWatch Metrics” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/monitoring-cloudwatch.html>. By combining logs and metrics, you can log errors and monitor your API’s performance.

Creating access logs

1. Create log group

```
aws logs create-log-group \
    --log-group-name my-log-group
```
2. Set up access logging on a stage

```
aws apigatewayv2 update-stage \
    --api-id abcdef \
    --stage-name '$default' \
    --access-log-settings '{"DestinationArn":
"arn:aws:logs:region:account-id:log-group:my-log-
group", "Format": "$context.identity.sourceIp - -
[$context.requestTime] \"$context.httpMethod
$context.routeKey $context.protocol\"
$context.status $context.responseLength
$context.requestId"}'
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

82

API Gateway creates the log groups and log streams for error logs. To use access logging, create the log group and then point a stage to that log group when you enable access logging on the stage.

You can choose from common formats; for example, common log format (CLF) and JSON. Examples of commonly used access log formats are shown in the API Gateway console, and are provided in the API Gateway Developer Guide. For more information, see the following pages:

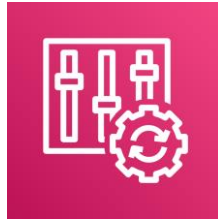
- “Setting Up CloudWatch Logging for a REST API” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/set-up-logging.html>
- “Configuring Logging for an HTTP API” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-logging.html>

Other monitoring tools



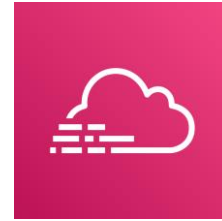
AWS X-Ray

Trace and analyze requests from end to end



AWS Config

Assess, audit, and evaluate configuration updates



AWS CloudTrail

Maintain a record of actions that a user, role, or AWS service takes on API Gateway



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

83

X-Ray answers the questions:

- What is the breakdown of this individual transaction as it passed through my application?
- Where are problems occurring across my distributed application?

With X-Ray, you can configure services to capture trace data about the requests. For some services including API Gateway, you can directly enable X-Ray.

X-Ray combines the trace data from each service involved in serving a request into a unit called a *trace*.

For developers, it provides visual detection of latency distribution and quick isolation of outliers and trends and makes it easier to debug application code. It also lets you filter and group requests by error type.

AWS Config answers the questions:

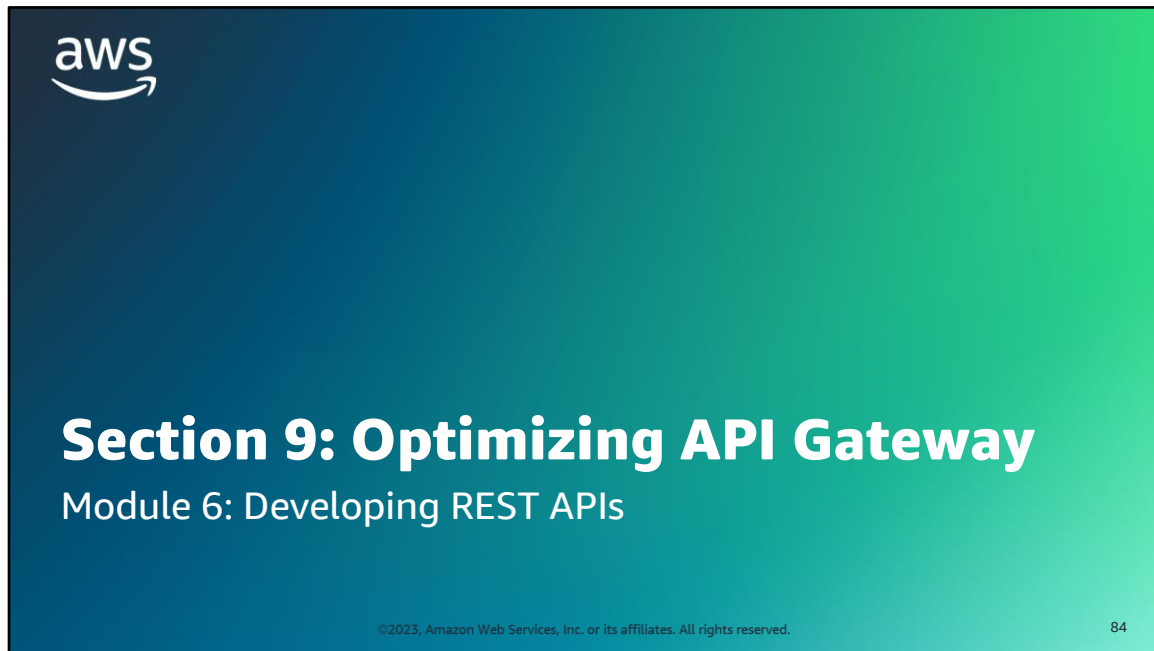
- Does this modification comply with our rules?
- How do these resources relate to other resources?

AWS Config provides a normalized snapshot of how your resources are configured and lets you create rules that enforce the compliant state of those resources. An AWS Config rule represents desired configuration settings for specific AWS resources or for an entire AWS account. If resources violate a rule, AWS Config flags this as noncompliant and notifies you through Amazon Simple Notification Service (Amazon SNS).

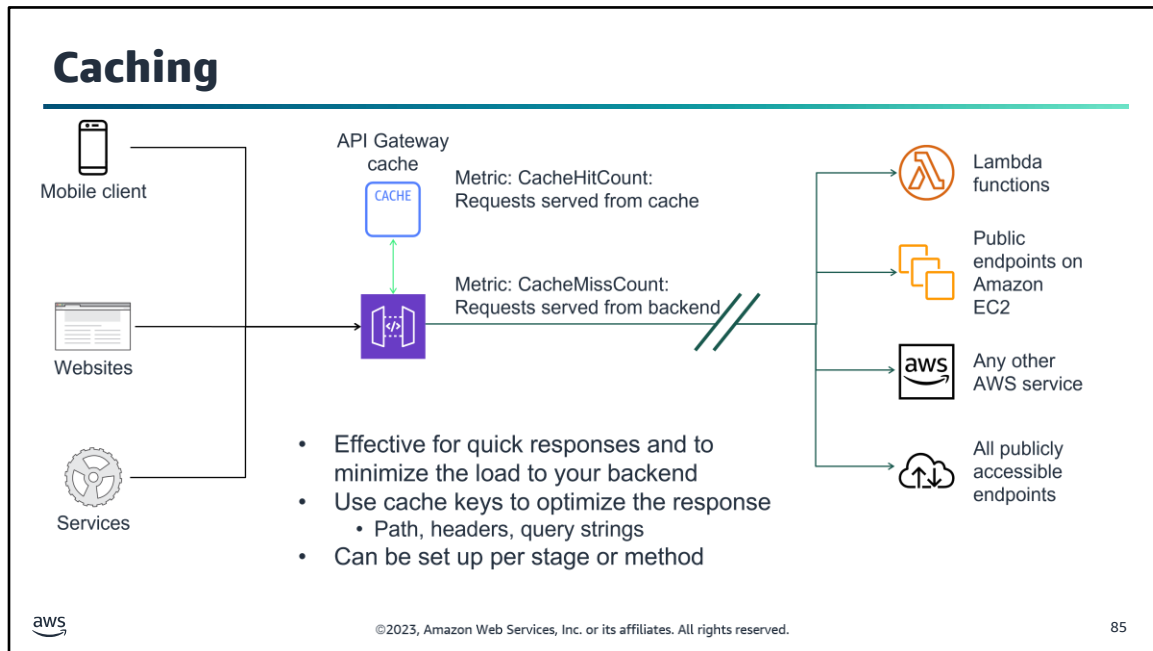
AWS CloudTrail answers the questions

- Which resources were modified?
- Who modified the resources?
- When were they modified?

CloudTrail is enabled when you create an account. When activity occurs in your AWS account, that activity is recorded in a *CloudTrail event*, and you can see recent events in the event history. The CloudTrail event history provides a viewable, searchable, and downloadable record of recent CloudTrail events. Details of API actions include the identity of the requestor, time of the API call, request parameters, and response elements returned by the service. Use this history to gain visibility into actions taken in your AWS account in the AWS Management Console, AWS SDKs, command line tools, and other AWS services.



Section 9: Optimizing API Gateway.



For REST APIs, you can set up API Gateway's cache and have it check the cache. It can check for a value that is requested in a GET request before trying to access the end target (for example, a Lambda function). With caching, you can reduce the number of calls that are made to your endpoint and also improve the latency of requests to your API.

If the value is in the cache, the value is returned to the client, and if not, the request is sent to the target. The new value is added to the cache, and then returned to the client. The next time that same request comes in, the value can be returned from the cache without hitting the target.

Values in the cache have a TTL setting that you can modify. You can enable data encryption of data that is stored in the cache.

Note that caching is priced hourly whether the cache is used or not.

API Gateway provides the following default metrics to help you understand how the cache is being used.

- *CacheHitCount* – Number of requests that were served from the API cache in a given period
- *CacheMissCount* – Number of requests that were served from the backend in a given period when caching is enabled

For information about how to enable API caching by using the API Gateway response cache, see "Enabling API Caching to Enhance Responsiveness" at <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-caching.html>.

Payload compression

Use payload compression to reduce the size of the client payload.
Choose a supported content coding (**deflate**, **gzip**, or **identity**).
Set a **minimumCompressionSize** to tell API Gateway when to apply compression to a payload.
Submit API requests with a compressed payload and an appropriate **Content-Encoding** header.
Test your payload settings to make sure that compression and decompression improve performance.

```
aws apigateway update-rest-api \  
  --rest-api-id 1234567890 \  
  --patch-operations  
op=replace,path=/minimumCompressionSize,value=0
```

Minimum compression
value of zero = compress
all payloads



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

86

Compression reduces the amount of data that is sent from API Gateway to clients, which can reduce costs and improve the performance of your APIs.

API Gateway allows your client to call your API with compressed payloads by using one of the supported content codings. For the list of supported content codings, see “Enable Payload Compression for an API” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-enable-compression.html>.

By default, API Gateway supports decompression of the method request payload. However, you must configure your API to enable compression of the method response payload.

You can also send compressed requests from the client.

You can update an existing API to use compression from the CLI by using:

```
aws apigateway update-rest-api \  
  --rest-api-id 1234567890 \  
  --patch-operations  
op=replace,path=/minimumCompressionSize,value=0
```

A minimum compression size of 0 means that all payloads will be compressed.

Compressing data of a small size might actually increase the final data size. Compression in API Gateway and decompression in the client might increase overall latency and require more computing times. You should run test cases against your API to determine an optimal value.

Compression example

Uncompressed
client payload

```
{  
  "type": "dog",  
  "price": 249.99  
}
```

Compressed
with gzip

```
   RPP*,HU RPJ OW  e&    L, , -y j
```

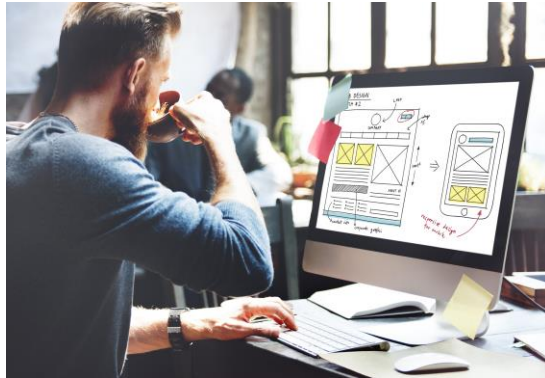


 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

87

This example shows what the payload looks like uncompressed and compressed with gzip.

Lab 6.1: Developing REST APIs with Amazon API Gateway



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

88

You will now complete Lab 6.1: Developing REST APIs with Amazon API Gateway.

Lab: Scenario

In the previous lab, you played the role of Sofía to build a web application for the café. As part of this process, you created a DynamoDB table named FoodProducts to store information about café menu items. You also loaded data into the table and configured code to extract unique items from the table.

In this lab, you continue to play the role of Sofía. You add the ability to query menu information from the database table through REST API calls. For the initial development, you use mock data endpoints.

In the next lab, you will replace the mock endpoints with real endpoints, which the web application will use to connect to the DynamoDB backend.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

89

In this lab, you will continue to play the role of Sofía. You add the ability to query menu information from the database table through REST API calls. For the initial development, you use mock data endpoints.

Lab: Tasks

1. Preparing the development environment
2. Creating the first API endpoint (GET)
3. Creating the second API endpoint (GET)
4. Creating the third API endpoint (POST)
5. Deploying the API
6. Updating the website to use the APIs



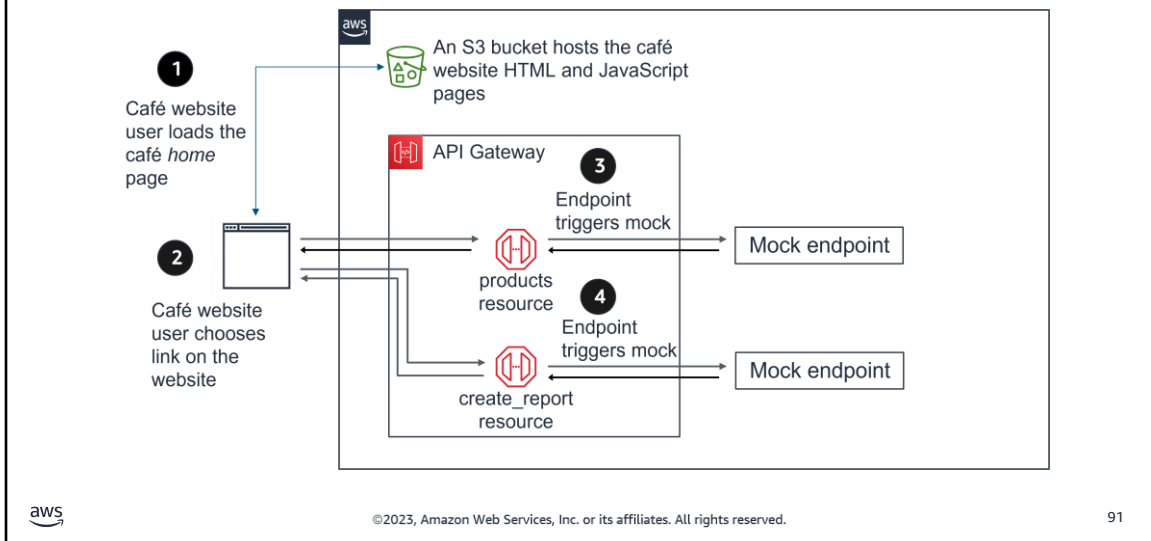
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

90

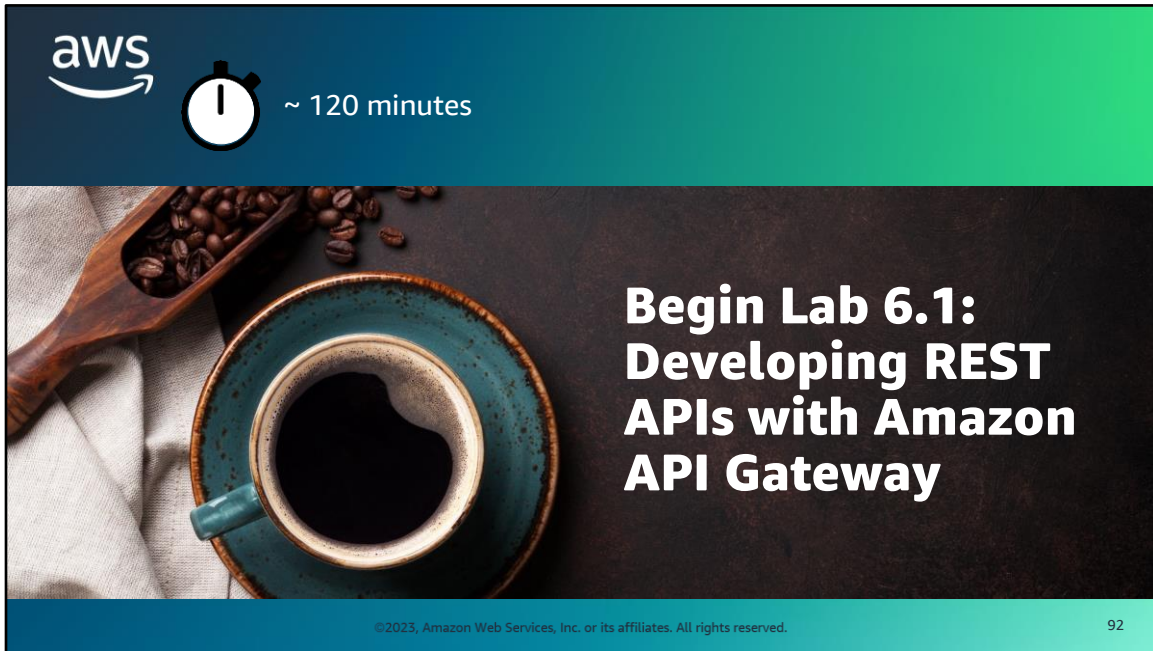
In this lab, you will complete the following tasks:

1. Preparing the development environment
2. Creating the first API endpoint (GET)
3. Creating the second API endpoint (GET)
4. Creating the third API endpoint (POST)
5. Deploying the API
6. Updating the website to use the APIs

Lab: Final product



The diagram summarizes the resources and architecture that you will build during the lab.



The image is a promotional banner for an AWS lab. It features a dark teal background with a green gradient at the top. In the top left corner, the AWS logo is displayed next to a clock icon and the text '~ 120 minutes'. The main visual is a photograph of a wooden spoon filled with coffee beans, with a few beans scattered on a dark surface next to a teal ceramic coffee cup. The text 'Begin Lab 6.1: Developing REST APIs with Amazon API Gateway' is written in white, bold, sans-serif font on the right side of the image. At the bottom, a teal bar contains the copyright notice '©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.' on the left and the number '92' on the right.

aws ~ 120 minutes

**Begin Lab 6.1:
Developing REST
APIs with Amazon
API Gateway**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 92

It is now time to start the lab.

Lab debrief: Key takeaways

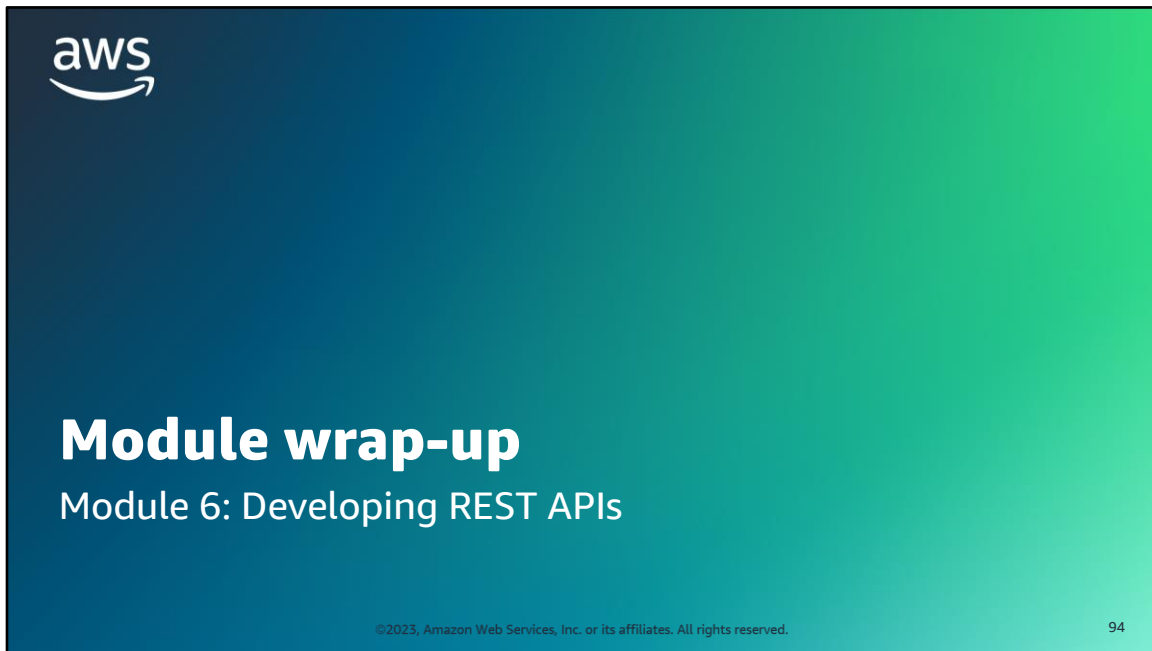


aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

93

Your educator might choose to lead a conversation about the key takeaways from this lab after you have completed it.



It's now time to review the module and wrap up with a knowledge check and discussion of a practice certification exam question.

Module summary

In summary, in this module, you learned how to do the following:

- Recognize APIs
- Describe API Gateway
- Indicate the steps for developing REST APIs with API Gateway
- Use API Gateway to create, publish, maintain, monitor, and secure APIs



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

95

In summary, in this module, you learned how to do the following:

- Recognize APIs
- Describe API Gateway
- Indicate the steps for developing REST APIs with API Gateway
- Use API Gateway to create, publish, maintain, monitor, and secure APIs

Complete the knowledge check



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

96

It is now time to complete the knowledge check for this module.

Sample exam question

A developer is building a web application that uses API Gateway. The developer wants to maintain different environments for development and production (dev and prod) workloads. The API will be backed by a Lambda function with two aliases: one for dev and one for prod.

How can the developer achieve this result with the LEAST amount of configuration?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- Different environments
- Dev and prod
- Lambda function with two aliases



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

97

It is important to fully understand the scenario and question being asked before even reading the answer choices. Find the keywords in this scenario and question that will help you find the correct answer.

Sample exam question: Responses

A developer is building a web application that uses API Gateway. The developer wants to maintain **different environments** for development and production (**dev and prod**) workloads. The API will be backed by a **Lambda function with two aliases**: one for dev and one for prod.

How can the developer achieve this result with the LEAST amount of configuration?

Choice	Response
A	Create a REST API for each environment, and integrate the APIs with the corresponding dev and prod aliases of the Lambda function. Then, deploy the two APIs to their respective stages, and access them by using the stage URLs.
B	Create one REST API, and integrate it with the Lambda function by using a stage variable in place of an alias. Then, deploy the API to two different stages—dev and prod—and create a stage variable in each stage with different aliases as the values. Access the API by using the different stage URLs.
C	Create one REST API, integrate it with the dev alias of the Lambda function, and deploy it to a dev environment. Configure a canary release deployment for prod where the canary will integrate with the Lambda prod alias.
D	Create one REST API, integrate it with the prod alias of the Lambda function, and deploy it to a prod environment. Configure a canary release deployment for dev where the canary will integrate with the Lambda dev alias.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

98

Now that we have bolded the keywords in this scenario, let us look at the answers.

Sample exam question: Answer

The correct answer is B.

Choice	Response
A	Create a REST API for each environment, and integrate the APIs with the corresponding dev and prod aliases of the Lambda function. Then, deploy the two APIs to their respective stages, and access them by using the stage URLs.
B	Create one REST API, and integrate it with the Lambda function by using a stage variable in place of an alias. Then, deploy the API to two different stages—dev and prod—and create a stage variable in each stage with different aliases as the values. Access the API by using the different stage URLs.
C	Create one REST API, integrate it with the dev alias of the Lambda function, and deploy it to a dev environment. Configure a canary release deployment for prod where the canary will integrate with the Lambda prod alias.
D	Create one REST API, integrate it with the prod alias of the Lambda function, and deploy it to a prod environment. Configure a canary release deployment for dev where the canary will integrate with the Lambda dev alias.

 ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 99

Look at the answer choices, and rule them out based on the keywords that were previously highlighted.

The correct answer is B. Create one REST API, and integrate it with the Lambda function by using a stage variable in place of an alias. Then, deploy the API to two different stages—dev and prod—and create a stage variable in each stage with different aliases as the values. Access the API by using the different stage URLs.

With deployment stages in API Gateway, users can manage multiple release stages for each API, such as alpha, beta, and production. Using stage variables that can be configured, an API deployment stage can interact with different backend endpoints. Users can use API Gateway stage variables to reference a single Lambda function with multiple versions and aliases.

Additional resources

- Videos: [Serverlessland.com videos about API Gateway](#)
- AWS whitepaper: [Security Overview of Amazon API Gateway](#)
- AWS Compute Blog: [Announcing HTTP APIs for Amazon API Gateway](#)
- FAQ: [Amazon API Gateway FAQs](#)

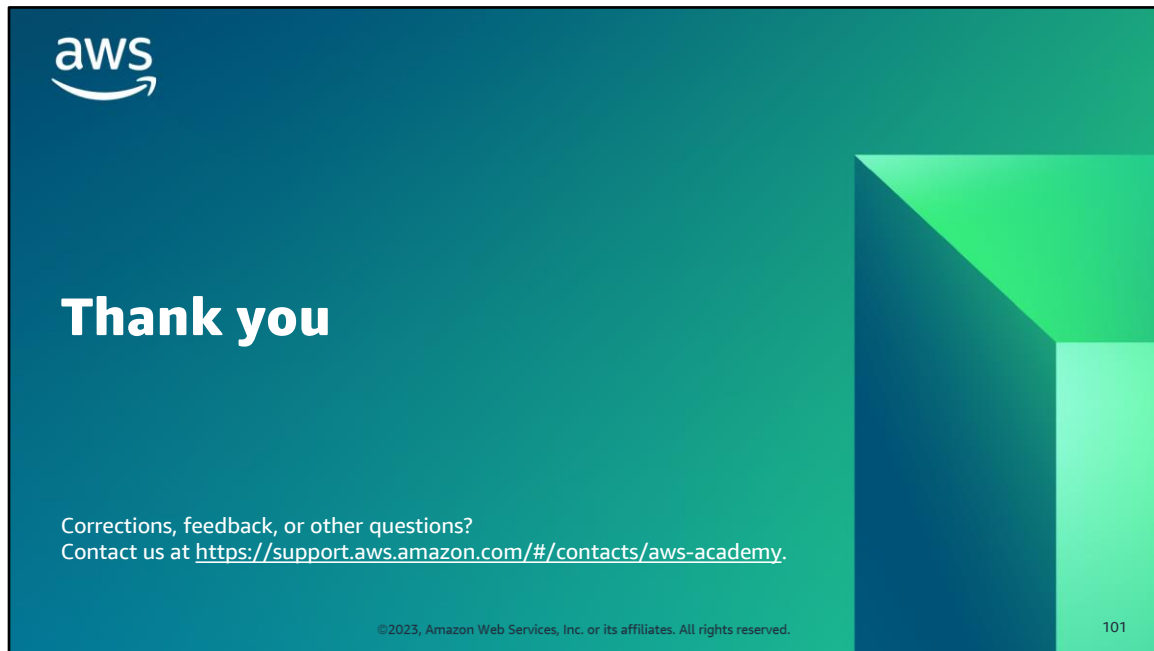


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

100

To learn more about the topics covered in this module, the following resources might be helpful:

- Videos: [Serverlessland.com videos about API Gateway](#) (<https://serverlessland.com/video?tag=Amazon%20API%20Gateway>)
- AWS whitepaper: [Security Overview of Amazon API Gateway](#) (<https://d1.awsstatic.com/whitepapers/api-gateway-security.pdf>)
- AWS Compute Blog: [Announcing HTTP APIs for Amazon API Gateway](#) (<https://aws.amazon.com/blogs/compute/announcing-http-apis-for-amazon-api-gateway/>)
- FAQ: [Amazon API Gateway FAQs](#) (<https://aws.amazon.com/api-gateway/faqs/>)



Thank you for completing this module.